MitoChime: A Machine Learning Pipeline for Detecting PCR-Induced Chimeras in Mitochondrial Illumina Reads

A Special Project Proposal
Presented to
the Faculty of the Division of Physical Sciences and Mathematics
College of Arts and Sciences
University of the Philippines Visayas
Miagao, Iloilo

In Partial Fulfillment
of the Requirements for the Degree of
Bachelor of Science in Computer Science

by

Duranne Duran
Yvonne Lin
Daniella Pailden

Adviser
Francis D. Dimzon, Ph.D.

April 12, 2026

Abstract

Next-generation sequencing (NGS) platforms have advanced research but remain susceptible to artifacts such as PCR-induced chimeras that compromise mitochondrial genome assembly. These artificial hybrid sequences are problematic for small, circular, and repetitive mitochondrial genomes, where they can generate fragmented contigs and false junctions. Existing detection tools, such as UCHIME, are optimized for amplicon-based microbial community analysis and depend on reference databases or abundance assumptions unsuitable for organellar assembly. To address this gap, this study presents MitoChime, a machine learning pipeline for detecting PCR-induced chimeric reads in *Sardinella lemuru* Illumina paired-end data without relying on external reference databases.

Using simulated datasets containing clean and chimeric reads, a feature set was extracted, combining alignment-based metrics (e.g., supplementary alignments, soft-clipping) with sequence-derived statistics (e.g., k-mer composition, microhomology). A comparative evaluation of supervised learning models identified tree-based ensembles CatBoost and Gradient Boosting as top performers, achieving an F1-score of 0.77 and an ROC-AUC of 0.84 on held-out test data. Feature importance analysis highlighted soft-clipping and k-mer compositional shifts as the strongest predictors of chimerism, whereas microhomology contributed minimally. Integrating MitoChime as a pre-assembly step can aid in streamlining mitochondrial reconstruction pipelines.

Keywords: Chimera detection, Mitochondrial genome, Assembly, Machine learning

Contents

1 Introduction **1**

1.1 Overview 1

1.2 Problem Statement 3

1.3 Research Objectives 3

1.3.1 General Objective 3

1.3.2 Specific Objectives 4

1.4 Scope and Limitations of the Research 4

1.5 Significance of the Research 6

2 Review of Related Literature **7**

2.1 The Mitochondrial Genome 7

2.1.1 Mitochondrial Genome Assembly 8

2.2 PCR Amplification and Chimera Formation 9

2.3 Existing Traditional Approaches for Chimera Detection 10

2.3.1 UCHIME 11

2.3.2 UCHIME2 12

2.3.3 CATch 13

2.3.4 ChimPipe 14

2.4 Machine Learning Approaches for Chimera and Sequence Quality Detection 15

2.4.1 Feature-Based Representations of Genomic Sequences . . . 15

2.5 Synthesis of Chimera Detection Approaches 16

3 Research Methodology **19**

3.1 Research Activities 19

3.2 Data Collection 21

3.3 Data Simulation and Preprocessing 21

3.3.1 Simulation of Clean Reads 21

3.3.2 Simulation of Chimeric Reads 22

3.3.3 Reference Indexing and Alignment 22

3.3.4 Construction of Labelled Datasets 23

3.4 Feature Extraction Pipeline 23

3.4.1 Supplementary Alignment and Alignment-Structure Features 24

3.4.2 Soft-Clipping and Breakpoint Features 24

3.4.3 K-mer Composition Discontinuity 24

3.4.4 Microhomology Features 25

3.4.5 Alignment Quality Features 25

3.5 Classical Machine Learning Model Development 25

3.6 Hyperparameter Optimization and Model Evaluation 26

3.7 Feature Importance and Ablation 27

3.8 Deep Learning Model Development 27

3.8.1 Input Preparation 28

3.8.2 Convolutional Neural Network 28

3.8.3 Bidirectional GRU Model 28

3.8.4 Training and Evaluation 29

3.9 Threshold-Based Read Filtering 29

3.10 Downstream Assembly Validation 29

3.11 Validation and Comparative Testing 30

3.12 Documentation and Reproducibility 30

3.13 Calendar of Activities 31

4 Results and Discussion **32**

4.1 Descriptive Analysis of Features 33

4.1.1 Summary Statistics Per Class 33

4.1.2 Correlation Analysis of Extracted Features 38

4.2 Baseline Classification Performance 39

4.3 Effect of Hyperparameter Tuning 41

4.3.1 Confusion Matrix and Error Patterns 43

4.3.2 ROC and Precision–Recall Curves 44

4.4 Feature Importance 45

4.4.1 Permutation Importance of Individual Features 45

4.4.2 Feature Family Importance 46

4.5 Feature Selection 47

4.5.1 Cumulative Importance Curve 47

4.5.2 Performance Comparison Across Feature Sets 48

4.5.3 Interpretation and Final Feature Set Choice 49

4.6 Convolutional Neural Network (CNN) Performance and Classification Results 50

4.7 Recurrent Neural Network (RNN) Performance and Classification Results 52

4.7.1 External Assembly Testing 55

4.7.2 Comparison Across Models 58

4.8 Summary of Findings 59

A Complete Per-Class Summary Statistics **61**

B Boxplots for All Numeric Features by Feature Family **65**

B.0.1 SA Structure (Supplementary Alignment and Segment Metrics) 65

B.0.2 Clipping-Based Features 67

B.0.3 K-mer Features 67

B.0.4 Microhomology Features 68

B.0.5 Others 68

List of Figures

3.1 Overall workflow of the study. 20

4.1 Boxplots of selected features for clean and chimeric reads. 37

4.2 Feature correlation heatmap showing relationships among alignment-derived and sequence-derived features. 39

4.3 Test F1 of all baseline classifiers. 41

4.4 Comparison of test F1 (left) and ROC–AUC (right) for baseline and tuned models. 42

4.5 Confusion matrix of the tuned Gradient Boosting classifier on the held-out test set. 44

4.6 ROC and precision–recall curves for the tuned Gradient Boosting classifier on the held-out test set. 45

4.7 Permutation-based feature importance for the tuned Gradient Boosting classifier. 46

4.8 Aggregated feature family importance for the tuned Gradient Boosting classifier. 47

4.9 Cumulative importance curve of features sorted by importance for the tuned Gradient Boosting classifier. 48

4.10 Comparison of F1 and ROC–AUC across feature set variants for the tuned Gradient Boosting classifier. 49

4.11 CNN training and validation loss and validation F1 across epochs. 51

4.12 Confusion matrix of the CNN model on the held-out test set. . . . 52

4.13 RNN-kmer BiGRU training and validation loss and validation F1 across epochs. 54

4.14 Confusion matrix of the RNN-kmer BiGRU model on the held-out test set. 55

4.15 Comparison of held-out test F1 and ROC–AUC for Gradient Boosting, CNN, and RNN models. 58

B.1 Boxplots of SA Structure features by class (1/2). 65

B.2 Boxplots of SA Structure features by class (2/2). 66

B.3 Boxplots of clipping-based features by class. 67

B.4 Boxplots of k-mer features by class. 67

B.5 Boxplots of microhomology features by class. 68

155 B.6 Boxplots of other numeric features by class. 68

List of Tables

2.1 Comparison of Chimera Detection Approaches and Tools 17

3.1 Timetable of activities. 31

4.1 Summary statistics of selected key features by class. 36

4.2 Performance of baseline classifiers on the held-out test set. 40

4.3 Performance of tuned classifiers on the held-out test set. 42

4.4 Test set performance of feature set variants using the tuned Gradient Boosting classifier. 49

4.5 Five-fold cross-validation performance of the CNN model. 50

4.6 Held-out test performance of the CNN model. 51

4.7 Five-fold cross-validation performance of the RNN-kmer BiGRU model. 53

4.8 Held-out test performance of the RNN-kmer BiGRU model. . . . 54

4.9 Comparison of held-out test performance across the top-performing models. 58

A.1 Complete per-class summary statistics for all extracted features. . 62

Chapter 1

Introduction

1.1 Overview

The rapid advancement of next-generation sequencing (NGS) technologies has transformed genomic research by enabling high-throughput and cost-effective DNA analysis (Metzker, 2010). Among current platforms, Illumina sequencing remains the most widely adopted, capable of producing millions of short reads that can be assembled into reference genomes or analyzed for genetic variation (Bentley et al., 2008; Glenn, 2011). Despite its high base-calling accuracy, Illumina sequencing is prone to artifacts introduced during library preparation, particularly polymerase chain reaction (PCR)-induced chimeras, which are artificial hybrid sequences that do not exist in the true genome (Judo, Wedel, & Wilson, 1998).

PCR chimeras form when incomplete extension products from one template

anneal to an unrelated DNA fragment and are extended, creating recombinant reads (Qiu et al., 2001). In mitochondrial genome assembly, such artifacts are especially problematic because the mitochondrial genome is small, circular, and often repetitive (Boore, 1999; Cameron, 2014). Even a small number of chimeric or misjoined reads can reduce assembly contiguity and introduce false junctions during organelle genome reconstruction (Dierckxsens, Mardulyn, & Smits, 2017; Hahn, Bachmann, & Chevreux, 2013; Jin et al., 2020). Existing assembly tools such as GetOrganelle and MITObim assume that input reads are largely free of such artifacts (Hahn et al., 2013; Jin et al., 2020). Consequently, undetected chimeras may produce fragmented assemblies or misidentified organellar boundaries. To ensure accurate reconstruction of mitochondrial genomes, a reliable method for detecting PCR-induced chimeras before assembly is essential.

This study focuses on mitochondrial sequencing data from the genus *Sardinella*, a group of small pelagic fishes widely distributed in Philippine waters. Among them, *Sardinella lemuru* (Bali sardinella) is one of the country's most abundant and economically important species, providing protein and livelihood to coastal communities (Labrador, Agmata, Palermo, Ravago-Gotanco, & Pante, 2021; Willette, Bognot, Mutia, & Santos, 2011). Accurate mitochondrial assemblies are critical for understanding its population genetics, stock structure, and evolutionary history. However, assembly pipelines often encounter errors or fail to complete due to undetected chimeric reads. To address this gap, this research introduces MitoChime, a machine learning pipeline designed to detect PCR-induced chimeric reads using both alignment-based and sequence-derived statistical features. The tool aims to provide bioinformatics laboratories, particularly the Philippine Genome Center Visayas (PGC Visayas), with an efficient

solution for improving mitochondrial genome reconstruction.

1.2 Problem Statement

Chimeric reads can distort assembly graphs and cause misassemblies, with particularly severe effects in mitochondrial genomes (Boore, 1999; Cameron, 2014). Existing assembly pipelines such as GetOrganelle, MITObim, and NOVOPlasty assume that sequencing reads are free of such artifacts (Dierckxsens et al., 2017; Hahn et al., 2013; Jin et al., 2020). At PGC Visayas, several mitochondrial assemblies have failed or yielded incomplete contigs despite sufficient coverage, suggesting that undetected chimeric reads compromise assembly reliability. Meanwhile, existing chimera detection tools such as UCHIME and VSEARCH were developed primarily for amplicon-based community analysis and rely heavily on reference or taxonomic comparisons (Edgar, Haas, Clemente, Quince, & Knight, 2011; Rognes, Flouri, Nichols, Quince, & Mahé, 2016). These approaches are unsuitable for single-species organellar data, where complete reference genomes are often unavailable.

1.3 Research Objectives

1.3.1 General Objective

This study aims to develop and evaluate a machine learning-based pipeline (MitoChime) that detects PCR-induced chimeric reads in *Sardinella lemuru* mito-

chondrial sequencing data in order to improve the quality and reliability of downstream mitochondrial genome assemblies.

1.3.2 Specific Objectives

Specifically, the study aims to:

1. construct simulated *Sardinella lemuru* Illumina paired-end datasets containing both clean and PCR-induced chimeric reads,
2. extract alignment-based and sequence-based features such as k-mer composition, junction complexity, and split-alignment counts from both clean and chimeric reads,
3. train, validate, and compare supervised machine learning models for classifying reads as clean or chimeric,
4. determine feature importance and identify indicators of PCR-induced chimerism,
5. integrate the optimized classifier into a modular and interpretable pipeline deployable on standard computing environments at PGC Visayas.

1.4 Scope and Limitations of the Research

This study focuses solely on PCR-induced chimeric reads in *Sardinella lemuru* mitochondrial sequencing data, with the species choice guided by four considerations: (1) to limit interspecific variation in mitochondrial genome size, GC

content, and repetitive regions so that differences in read patterns can be attributed more directly to PCR-induced chimerism, (2) to align the analysis with relevant *S. lemuru* sequencing projects at PGC Visayas, (3) to take advantage of the availability of *S. lemuru* mitochondrial assemblies and raw datasets in public repositories such as the National Center for Biotechnology Information (NCBI), which facilitates reference selection and benchmarking, and (4) to develop a tool that directly supports local studies on *S. lemuru* population structure and fisheries management.

The study emphasizes `wgsim`-based simulations and selected empirical mitochondrial datasets from *S. lemuru*. It excludes naturally occurring chimeras, nuclear mitochondrial pseudogenes (NUMTs), and large-scale assembly rearrangements in nuclear genomes. Feature extraction is restricted to low-dimensional alignment and sequence statistics, such as k-mer frequency profiles, GC content, soft and hard clipping metrics, and split-alignment counts rather than high-dimensional deep learning embeddings. This design keeps model behaviour interpretable and ensures that the pipeline can be run on standard workstations at PGC Visayas. Testing on long-read platforms (e.g., Nanopore, PacBio) and other taxa is outside the scope of this project.

Other limitations in this study include the following: simulations with varying error rates were not performed, so the effect of different sequencing errors on model performance remains unexplored; alternative parameter settings, including k-mer lengths and microhomology window sizes, were not systematically tested, which could affect the sensitivity of both k-mer and microhomology feature detection; and the machine learning models rely on supervised training with labeled examples, which may limit their ability to detect novel or unexpected chimeric

patterns.

1.5 Significance of the Research

This research provides both methodological and practical contributions to mitochondrial genomics and bioinformatics. First, MitoChime detects PCR-induced chimeric reads prior to genome assembly, with the goal of improving the contiguity and correctness of *Sardinella lemuru* mitochondrial assemblies. Second, it replaces informal manual curation with a documented workflow, improving automation and reproducibility. Third, the pipeline is designed to run on computing infrastructures commonly available in regional laboratories, enabling routine use at facilities such as PGC Visayas. Finally, more reliable mitochondrial assemblies for *S. lemuru* provide a stronger basis for downstream applications in the field of fisheries and genomics.

Chapter 2

Review of Related Literature

This chapter presents an overview of the literature relevant to the study. It discusses the biological and computational foundations underlying mitochondrial genome analysis and assembly, as well as existing tools, algorithms, and techniques related to chimera detection and genome quality assessment. The chapter aims to highlight the strengths, limitations, and research gaps in current approaches that motivate the development of the present study.

2.1 The Mitochondrial Genome

Mitochondrial genome (mtDNA) is a small, typically circular molecule found in most eukaryotes. It encodes essential genes involved in oxidative phosphorylation and energy metabolism. Because of its conserved structure, mtDNA has become a valuable genetic marker for studies in population genetics and phylogenetics (Anderson et al., 1981; Boore, 1999). In animal species, the mitochondrial genome

ranges from 15–20 kilobase and contains 13 protein-coding genes, 22 tRNAs, and two rRNAs arranged compactly without introns (Gray, 2012). In comparison to nuclear DNA, the ratio of the number of copies of mtDNA is higher and has simple organization which make it particularly suitable for genome sequencing and assembly studies (Dierckxsens et al., 2017).

2.1.1 Mitochondrial Genome Assembly

Mitochondrial genome assembly refers to the reconstruction of the complete mitochondrial DNA (mtDNA) sequence from raw or fragmented sequencing reads. It is conducted to obtain high-quality, continuous representations of the mitochondrial genome that can be used for a wide range of analyses, including species identification, phylogenetic reconstruction, evolutionary studies, and investigations of mitochondrial diseases. Because mtDNA evolves rapidly, its assembled sequence provides valuable insights into population structure, lineage divergence, and adaptive evolution across taxa (Boore, 1999). Compared to nuclear genome assembly, assembling the mitochondrial genome is often considered more straightforward but still encounters technical challenges such as the formation of chimeric reads. Commonly used tools for mitogenome assembly such as GetOrganelle and MITObim operate under the assumption of organelle genome circularity, and are vulnerable when chimeric reads disrupt this circular structure, resulting in assembly errors (Hahn et al., 2013; Jin et al., 2020).

2.2 PCR Amplification and Chimera Formation

PCR plays an important role in NGS library preparation, as it amplifies target DNA fragments for downstream analysis. However as previously mentioned, the amplification process can also introduce chimeric reads which compromises the quality of the input reads supplied to sequencing or assembly workflows. Chimeras typically arise when incomplete extension occurs during a PCR cycle. This causes the DNA polymerase to switch from one template to another and generate hybrid recombinant molecules (Judo et al., 1998). Artificial chimeras are produced through such amplification errors, whereas biological chimeras occur naturally through genomic rearrangements or transcriptional events.

In the context of amplicon-based sequencing, the presence of chimeras can inflate estimates of genetic or microbial diversity and may cause misassemblies during genome reconstruction. Qin et al. (2023) has reported that chimeric sequences may account for more than 10% of raw reads in amplicon datasets. This artifact tends to be most prominent among rare operational taxonomic units (OTUs) or singletons, which are sometimes misinterpreted as novel diversity, further causing the complication of microbial diversity analyses (Gonzalez, Zimmermann, & Saiz-Jimenez, 2004). As such, determining and minimizing PCR-induced chimera formation is vital for improving the quality of mitochondrial genome assemblies, and ensuring the reliability of amplicon sequencing data.

2.3 Existing Traditional Approaches for Chimera Detection

Several computational tools have been developed to identify chimeric sequences in NGS datasets. These tools generally fall into two categories: reference-based and de novo approaches. Reference-based chimera detection, also known as database-dependent detection, is one of the earliest and most widely used computational strategies for identifying chimeric sequences in amplicon-based community studies. These methods rely on the comparison of each query sequence against a curated, high-quality database of known, non-chimeric reference sequences (Edgar et al., 2011).

On the other hand, the de novo chimera detection, also referred to as reference-free detection, represents an alternative computational paradigm that identifies chimeric sequences without reliance on external reference databases. This method infer chimeras based on internal relationships among the sequences present within the dataset itself, making it particularly advantageous in studies of under explored or taxonomically diverse communities where comprehensive reference databases are unavailable or incomplete (Edgar, 2016; Edgar et al., 2011). The underlying assumption on this method is that during PCR, true biological sequences are generally more abundant as they are amplified early and dominate the read pool, whereas chimeric sequences appear later and are generally less abundant. The de novo approach leverage this abundance hierarchy, treating the most abundant sequences as supposed parents and testing whether less abundant sequences can be reconstructed as mosaics of these templates. Compositional and structural similarity are also evaluated to check whether different regions of a candidate

sequence correspond to distinct high-abundance sequences.

In practice, many modern bioinformatics pipelines combine both paradigms sequentially: an initial de novo step identifies dataset-specific chimeras, followed by a reference-based pass that removes remaining artifacts relative to established databases (Edgar, 2016). These two methods of detection form the foundation of tools such as UCHIME and later UCHIME2.

2.3.1 UCHIME

UCHIME is one of the most widely used tools for detecting chimeric sequences in amplicon-based studies and remains a standard quality-control step in microbial community analysis. Its core strategy is to test whether a query sequence (Q) can be explained as a mosaic of two parent sequences, (A and B), and to score this relationship using a structured alignment model (Edgar et al., 2011).

In reference mode, UCHIME divides the query into several segments and maps them against a curated database of non-chimeric sequences. Candidate parents are identified, and a three-way alignment is constructed. The algorithm assigns "Yes" votes when different segments of the query match different parents and "No" votes when the alignment contradicts a chimeric pattern. The final score reflects the balance of these votes. In de novo mode, UCHIME operationalizes the abundance-skew principle described earlier: high-abundance sequences are treated as candidate parents, and lower-abundance sequences are evaluated as potential mosaics. This makes the method especially useful when no reliable reference database exists.

Although UCHIME is highly sensitive, it faces key constraints. Chimeras formed from parents with very low divergence (below 0.8%) are difficult to detect because they are nearly indistinguishable from sequencing errors. Accuracy in reference mode depends strongly on database completeness, while de novo detection assumes that true parents are both present and sufficiently more abundant, such conditions are not always met.

2.3.2 UCHIME2

UCHIME2 extends the original algorithm with refinements tailored for high-resolution sequencing data. One of its major contributions is a re-evaluation of benchmarking practices. Edgar (2016) demonstrated that earlier accuracy estimates for chimera detection were overly optimistic because they relied on unrealistic scenarios where all true parent sequences were assumed to be present. Using the more rigorous CHSIMA benchmark, UCHIME2 showed the prevalence of "fake models" or real biological sequences that can be perfectly reconstructed as apparent chimeras of other sequences, which suggests that perfect chimera detection is theoretically unattainable. UCHIME2 also introduces several preset modes (e.g., denoised, balanced, sensitive, specific, high-confidence) designed to tune sensitivity and specificity depending on dataset characteristics. These modes allow users to adjust the algorithm to the expected noise level or analytical goals.

Despite these improvements, UCHIME2 must be applied with caution. The website manual explicitly advises against using UCHIME2 as a standalone chimera-filtering step in OTU clustering or denoising workflows because doing so can inflate both false positives and false negatives (Edgar, n.d.).

2.3.3 CATch

As previously mentioned, UCHIME (Edgar et al., 2011) relied on alignment-based sequences in amplicon data. However, researchers soon observed that different algorithms often produced inconsistent predictions. A sequence might be identified as chimeric by one tool but classified as non-chimeric by another, resulting in unreliable filtering outcomes across studies.

To address these inconsistencies, Mysara, Saeys, Leys, Raes, and Monsieurs (2015) developed the Classifier for Amplicon Tool Chimeras (CATCh), which represents the first ensemble machine learning system designed for chimera detection in 16S rRNA amplicon sequencing. Rather than depending on a single detection strategy, CATCh integrates the outputs of several established tools, including UCHIME, ChimeraSlayer, DECIPHER, Pintail, and Perseus. The individual scores and binary decisions generated by these tools are used as input features for a supervised learning model. The algorithm employs a Support Vector Machine (SVM) with a Pearson VII Universal Kernel (PUK) to determine optimal weightings among the input features and to assign each sequence a probability of being chimeric.

Benchmarking in both reference-based and de novo modes demonstrated significant performance improvements. CATCh achieved sensitivities of approximately 85 percent in reference-based mode and 92 percent in de novo mode, with corresponding specificities of approximately 96 percent and 95 percent. These results indicate that CATCh detected 7 to 12 percent more chimeras than any individual algorithm while maintaining high precision.

2.3.4 ChimPipe

Among the available tools for chimera detection, ChimPipe is a pipeline developed to identify chimeric sequences such as biological chimeras. It uses both discordant paired-end reads and split-read alignments to improve the accuracy and sensitivity of detecting biological chimeras (Rodriguez-Martin et al., 2017). By combining these two sources of information, ChimPipe achieves better precision than methods that depend on a single type of indicator.

The pipeline works with many eukaryotic species that have available genome and annotation data (Rodriguez-Martin et al., 2017). It can also predict multiple isoforms for each gene pair and identify breakpoint coordinates that are useful for reconstructing and verifying chimeric transcripts. Tests using both simulated and real datasets have shown that ChimPipe maintains high accuracy and reliable performance.

ChimPipe lets users adjust parameters to fit different sequencing protocols or organism characteristics. Experimental results have confirmed that many chimeric transcripts detected by the tool correspond to functional fusion proteins, demonstrating its utility for understanding chimera biology and its potential applications in disease research (Rodriguez-Martin et al., 2017).

2.4 Machine Learning Approaches for Chimera and Sequence Quality Detection

Traditional chimera detection tools rely primarily on heuristic or alignment-based rules. Recent advances in machine learning (ML) have demonstrated that models trained on sequence-derived features can effectively capture compositional and structural patterns in biological sequences. Although most existing ML systems such as those used for antibiotic resistance prediction, taxonomic classification, or viral identification are not specifically designed for chimera detection, they highlight how data-driven models can outperform similarity-based heuristics by learning intrinsic sequence signatures. In principle, ML frameworks can integrate indicators such as k-mer frequencies, GC-content variation and split-alignment metrics to identify subtle anomalies that may indicate a chimeric origin (Arango et al., 2018; Liang, Bible, Liu, Zou, & Wei, 2020; Ren et al., 2020).

2.4.1 Feature-Based Representations of Genomic Sequences

Feature extraction converts DNA sequences into numerical representations suitable for machine learning models. One approach is k-mer frequency analysis, which counts short nucleotide sequences within a read (Vervier, Mahé, Tournoud, Veyrieras, & Vert, 2015). High-frequency k-mers, including simple repeats such as "AAAAAA," can highlight repetitive or unusual regions that may occur near chimeric junctions. Comparing k-mer patterns across adjacent parts of a read can help identify such regions, while GC content provides an additional descriptor of

local sequence composition (Ren et al., 2020).

Alignment-derived features further inform junction detection. Long-read tools such as Sniffles (Sedlazeck et al., 2018) use split alignments to locate breakpoints across extended sequences, whereas short-read aligners like Minimap2 (Li, 2018) report supplementary and secondary alignments that indicate local discontinuities. Split alignments, where parts of a read map to different regions, can reveal template-switching events. These features complement k-mer profiles and enhance detection of potentially chimeric reads, even in datasets with incomplete references.

Microhomology, or short sequences shared between adjacent segments, is another biologically meaningful feature. Short microhomologies, typically 3–20 bp, are involved in template switching both in cellular repair pathways and during PCR, where they act as signatures of chimera formation (Peccoud et al., 2018; Sfeir & Symington, 2015). In PCR-induced chimeras, short identical sequences at junctions provide a clear signature of chimerism. Measuring the longest exact overlap at each breakpoint complements k-mer and alignment features and helps identify reads that are potentially chimeric.

2.5 Synthesis of Chimera Detection Approaches

To provide an integrated overview of the literature discussed in this chapter, Table 2.1 summarizes the major chimera detection studies, their methodological approaches, and their known limitations.

Table 2.1: Comparison of Chimera Detection Approaches and Tools

Method / Tool	Core Approach	Key Limitations
Reference-based Detection	Compares each query sequence against curated databases of verified, non-chimeric sequences; evaluates segment similarity to identify mosaic patterns.	Accuracy depends on database completeness; performs poorly for novel taxa or missing parents; limited sensitivity for low-divergence chimeras.
De novo Detection	Identifies chimeras using only internal dataset structure; leverages abundance hierarchy and compositional similarity to infer whether low-abundance sequences can be reconstructed from abundant parents.	Assumes true sequences are more abundant; fails when amplification bias distorts abundances; struggles when parental sequences are similarly abundant or highly similar.
UCHIME	Alignment-based model that partitions the query into segments, identifies parent candidates, and computes a chimera score via a three-way alignment; supports reference and de novo modes.	Reduced accuracy for very closely related parents (<0.8% divergence); sensitive to incomplete databases; de novo mode fails if parents are absent or not sufficiently more abundant.
UCHIME2	Updated UCHIME with improved benchmarking (CHSIMA) and multiple sensitivity/specificity presets; better handles incomplete references and dataset variability.	"Fake models" limit theoretical accuracy; genuine variants may mimic chimeras; not recommended as a standalone step in OTU or denoising pipelines due to increased false positives/negatives.
CATCh	First ensemble ML model for 16S chimera detection; integrates outputs of UCHIME, ChimeraSlayer, DECIPHER, Pintail, and Perseus using an SVM to boost overall prediction accuracy.	Performance constrained by underlying tools; ML model cannot capture features not present in component algorithms; may misclassify in highly novel or low-coverage datasets.
ChimPipe	Pipeline for detecting biological chimeras in RNA-seq using discordant paired-end reads and split-read alignments; identifies isoforms and breakpoint coordinates.	Requires high-quality genome and annotation; tailored to RNA-seq rather than amplicons; computationally intensive; limited to organisms with available reference genomes.

Across existing studies, no single approach reliably detects all forms of chimeric sequences, and the reviewed literature consistently shows that chimeras remain a persistent challenge in genomics and bioinformatics. Although the surveyed tools are not designed specifically for organelle genome assembly, they provide valuable insights into which methodological strategies are effective and where current approaches fall short. These limitations collectively define a clear research gap: the need for a specialized, feature-driven detection framework tailored to PCR-induced mitochondrial chimeras. Addressing this gap aligns with the research objective outlined in Section 1.3, which is to develop and evaluate a machine learning-based pipeline (MitoChime) that improves the quality of downstream mitochondrial genome assembly. In support of this aim, the subsequent chapters describe the design, implementation, and evaluation of the proposed tool.

Chapter 3

Research Methodology

This chapter describes the methodology used in developing and evaluating *MitoChime*, a machine learning pipeline for detecting PCR-induced chimeric reads in mitochondrial Illumina data. It covers data collection, read simulation, preprocessing and alignment, feature extraction, classical and deep learning model development, feature importance analysis, ablation testing, threshold-based filtering, downstream assembly validation, and documentation.

3.1 Research Activities

Figure 3.1 summarizes the overall workflow of the study. The process began with the collection of a mitochondrial reference genome of *Sardinella lemuru* from the National Center for Biotechnology Information (NCBI). This reference was used to simulate both clean and chimeric paired-end Illumina reads. The simulated reads were then aligned to the reference genome, and read-level features associated with

chimeric structure were extracted.

These features were used to train and evaluate classical machine learning models for chimera detection. The best-performing classical model, Gradient Boosting, was further examined through feature importance and ablation analyses. Subsequently, sequence-based deep learning models were also developed to assess whether chimeric reads could be identified directly from nucleotide sequences without relying on alignment-derived features. Finally, model predictions were used to filter suspected chimeric reads, and the effect of filtering was assessed through downstream mitochondrial assembly.

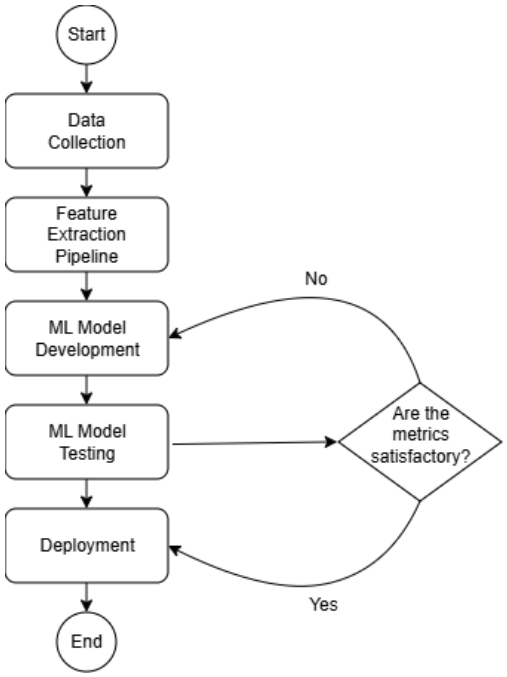


Figure 3.1: Overall workflow of the study.

3.2 Data Collection

The mitochondrial reference genome of *Sardinella lemuru* was obtained from the NCBI database in FASTA format (accession number `NC_039553.1`). This sequence served as the reference for simulation and alignment throughout the study.

Because experimentally validated chimeric mitochondrial read datasets were unavailable, simulated data were generated to produce labelled examples of both clean and chimeric reads for supervised learning.

3.3 Data Simulation and Preprocessing

3.3.1 Simulation of Clean Reads

Clean paired-end reads were simulated from the reference genome using `wgsim`. Reads of length 150 bp were generated to approximate Illumina short-read sequencing data. A total of 10,000 paired-end fragments were simulated using the following command format:

```
wgsim -1 150 -2 150 -r 0 -R 0 -X 0 -e 0.05 -N 10000 \
       original_reference.fasta clean_R1.fastq clean_R2.fastq
```

3.3.2 Simulation of Chimeric Reads

Chimeric templates were generated using a custom Python script. Two non-adjacent segments from the mitochondrial reference were selected and concatenated to mimic PCR-induced template switching. Short overlapping regions were optionally retained to represent microhomology near the junction.

The resulting chimeric templates were written to a FASTA file and used as input to `wgsim` under the same sequencing parameters applied to clean reads, producing 10,000 paired-end chimeric fragments.

3.3.3 Reference Indexing and Alignment

Both clean and chimeric reads were aligned to the original mitochondrial reference using `minimap2`. The reference was first indexed:

```
minimap2 -d ref.mmi original_reference.fasta
```

The simulated reads were then mapped using the short-read preset:

```
minimap2 -ax sr -t 8 ref.mmi clean_R1.fastq clean_R2.fastq > clean.sam
minimap2 -ax sr -t 8 ref.mmi chimera_R1.fastq chimera_R2.fastq > chimera.sam
```

The SAM files were converted to BAM format, sorted, and indexed using `samtools`:

```
samtools view -bS clean.sam -o clean.bam
```

```
samtools view -bS chimera.sam -o chimera.bam

samtools sort clean.bam -o clean.sorted.bam
samtools index clean.sorted.bam

samtools sort chimera.bam -o chimera.sorted.bam
samtools index chimera.sorted.bam
```

Only mapped primary reads were retained for alignment-based feature extraction.

3.3.4 Construction of Labelled Datasets

After preprocessing, reads were assigned class labels and organized into datasets for modelling. Clean reads were labelled as class 0, whereas chimeric reads were labelled as class 1. Separate datasets were prepared for feature-based machine learning and sequence-based deep learning experiments.

3.4 Feature Extraction Pipeline

A custom feature extraction pipeline was implemented in Python to compute read-level descriptors associated with PCR-induced chimerism from the BAM files. Each row of the resulting TSV matrix represented one read, and each column represented one numeric feature.

The pipeline focused on five feature groups: (1) supplementary alignment and

alignment-structure features, (2) soft-clipping and inferred breakpoint features, (3) k-mer composition discontinuity features, (4) microhomology features, and (5) alignment quality features.

3.4.1 Supplementary Alignment and Alignment-Structure Features

For each primary read, the pipeline checked for the presence of the `SA` tag. If present, the tag was parsed to recover supplementary alignment information, including mapped position, strand, mapping quality, and mismatch count. From this, features such as `has_sa`, `sa_count`, number of alignment segments, strand consistency, and summary statistics of segment distances and mapping qualities were computed.

3.4.2 Soft-Clipping and Breakpoint Features

Soft-clipped bases were extracted from the CIGAR string. Left and right soft-clipped lengths, as well as total clipped bases, were recorded. A candidate breakpoint position was inferred using clipping boundaries where available, or alignment/read midpoints otherwise.

3.4.3 K-mer Composition Discontinuity

Each read was divided into left and right segments based on the inferred breakpoint. Normalized k-mer frequency profiles were computed for both segments,

excluding ambiguous bases. Compositional differences were quantified using cosine difference and Jensen–Shannon divergence.

3.4.4 Microhomology Features

A local window surrounding the inferred breakpoint was examined to identify the longest exact suffix–prefix overlap between the left and right segments. The overlap length was recorded as `microhomology_length`, and its GC content as `microhomology_gc`.

3.4.5 Alignment Quality Features

General alignment quality measures, including mapping quality (`mapq`), were also included as predictors.

3.5 Classical Machine Learning Model Development

The extracted clean and chimeric feature matrices were merged into a single labelled dataset containing 23 numeric features. The dataset was partitioned into training (80%) and test (20%) subsets using stratified sampling.

A benchmarking stage was conducted using multiple classification algorithms, including Logistic Regression, linear support vector machine (SVM),

k-Nearest Neighbours, Gaussian Naïve Bayes, Random Forest, ExtraTrees, Bagging, Gradient Boosting, XGBoost, LightGBM, CatBoost, Multilayer Perceptron, and a dummy baseline. Model development was performed in Python using `scikit-learn` and related libraries.

Five-fold stratified cross-validation was used on the training set. Performance was assessed using accuracy, precision, recall, F1-score for the chimeric class, and the area under the receiver operating characteristic curve (ROC–AUC).

3.6 Hyperparameter Optimization and Model Evaluation

Top-performing model families from the initial benchmark were selected for further optimization using randomized search with five-fold stratified cross-validation. Hyperparameters were tuned according to model type, including the number of estimators, depth, learning rate, regularization, and hidden-layer configuration where applicable.

The best configuration for each model was selected based on mean cross-validation F1-score for the chimeric class. Tuned models were then retrained on the full training set and evaluated on the held-out test set. Final evaluation included accuracy, precision, recall, F1-score, ROC–AUC, and confusion matrices.

3.7 Feature Importance and Ablation

Feature importance analysis was performed using the Gradient Boosting model, which was identified as the best-performing classical machine learning model in the study. Two complementary approaches were applied: the model's built-in feature importance scores and permutation importance computed on the held-out test set. To aid interpretation, features were also organized into conceptual families reflecting alignment structure, soft clipping, k-mer discontinuity, microhomology, and alignment quality.

An ablation analysis was also performed by removing the microhomology-related features and retraining the Gradient Boosting model. Comparison of the full model and the microhomology-ablated variant enabled assessment of the specific contribution of microhomology to overall predictive performance.

3.8 Deep Learning Model Development

In addition to feature-based models, sequence-based deep learning models were developed to classify reads directly from nucleotide sequence. Two architectures were implemented: a one-dimensional convolutional neural network (CNN) and a bidirectional gated recurrent unit (BiGRU) model.

3.8.1 Input Preparation

For deep learning, read sequences were represented at a fixed length of 150 bp, consistent with the simulation settings used in the study.

For the CNN, each nucleotide sequence was transformed into a one-hot encoded representation, where each base was encoded as a binary vector. For the BiGRU, sequences were converted into overlapping k-mer tokens and encoded as integer sequences suitable for embedding-based sequence modelling.

3.8.2 Convolutional Neural Network

The CNN was implemented in PyTorch as a one-dimensional sequence classifier. The architecture consisted of stacked one-dimensional convolutional layers followed by nonlinear activation and pooling operations, then fully connected layers for binary classification. This model was designed to capture local sequence motifs and short-range compositional patterns that may indicate chimeric junctions.

3.8.3 Bidirectional GRU Model

The BiGRU model was likewise implemented in PyTorch. Tokenized k-mer sequences were first passed through an embedding layer, after which a bidirectional gated recurrent unit encoder was used to model sequential dependencies in both forward and reverse directions. The encoded representation was then passed to a classification head to predict whether a read was clean or chimeric.

3.8.4 Training and Evaluation

Both deep learning models were trained on the same training split used for the classical machine learning experiments and evaluated on the same held-out test set. Model training was performed using mini-batch optimization for multiple epochs, and the best-performing model checkpoint was selected based on validation performance, with emphasis on the F1-score of the chimeric class. Final deep learning evaluation also included accuracy, precision, recall, F1-score, and ROC–AUC.

3.9 Threshold-Based Read Filtering

The final trained models were applied as filtering tools by assigning each read a probability of belonging to the chimeric class. Reads exceeding a selected threshold were marked for removal. Read identifiers predicted as chimeric were collected, and both mates of each flagged pair were removed from the FASTQ files to preserve pairing structure.

Different probability thresholds were examined to assess the trade-off between chimera removal and retention of clean reads.

3.10 Downstream Assembly Validation

To evaluate the practical value of filtering, downstream mitochondrial assemblies were generated from both unfiltered and filtered read sets. Assembly results were

compared using metrics such as number of contigs, maximum contig length, and graph complexity.

Assemblies were produced using `SPAdes`. Improvement in assembly quality after filtering was interpreted as evidence that the removed reads were disruptive to mitochondrial reconstruction.

3.11 Validation and Comparative Testing

Validation was conducted using both internal and external evaluation. Internal validation was achieved through five-fold stratified cross-validation on the training data, while external evaluation was performed on the held-out test set.

Comparative testing was performed across classical machine learning and deep learning models to determine which approach provided the best balance of predictive accuracy, interpretability, and downstream utility.

3.12 Documentation and Reproducibility

All scripts, datasets, model outputs, and evaluation results were organized and version-controlled in a GitHub repository. Separate directories were maintained for references, simulated reads, alignments, extracted features, trained models, reports, and filtered outputs.

The computational environment was standardized using Conda, with dependencies recorded in an `environment.yml` file. Analytical procedures, software

versions, and command-line parameters were documented in the project `README`. Manuscript preparation was carried out using LaTeX.

3.13 Calendar of Activities

Table 3.1 presents the overall timetable of activities for the study.

Table 3.1: Timetable of activities.

Activities (2025–2026)	**Nov**	**Dec**	**Jan**	**Feb**	**Mar**	**Apr**	**May**
Data Collection and Simulation	●●●●						
Alignment and Feature Extraction	●	●●●					
Classical ML Development and Benchmarking		●	●●●	●●●			
Hyperparameter Tuning, Feature Importance, and Ablation				●	●●●	●	
Deep Learning Development			●	●●●	●●		
Filtering and Assembly Validation					●	●●●	●
Documentation and Manuscript Writing	●●●●	●●●●	●●●●	●●●●	●●●●	●●●●	●●●●

Chapter 4

Results and Discussion

This chapter presents the performance of the proposed feature set and machine learning models for detecting PCR-induced chimeric reads in simulated mitochondrial Illumina data. The behaviour of the extracted features is first examined through descriptive and correlation analyses, followed by a comparison of baseline and tuned classifiers. The chapter then examines model performance in detail and investigates the contribution of individual features and feature families, including the impact of feature selection on classification performance.

The final dataset contained 31,986 reads for training and 7,997 reads for testing, with classes balanced (approximately 4,000 clean and 4,000 chimeric reads in the test split).

4.1 Descriptive Analysis of Features

4.1.1 Summary Statistics Per Class

Summary statistics were computed separately for clean reads (class 0) and chimeric reads (class 1) to characterize the distributional behavior of the features. For each feature, the mean, standard deviation, median, first and third quartiles (Q1, Q3), interquartile range (IQR), minimum, maximum, and sample size (n) were calculated.

Only a subset of the features is summarized in the main text to highlight key trends, and not all summary statistics columns are shown for brevity. The complete set of per-class summary statistics for all features is provided in Appendix A (Table A.1).

Alignment and Supplementary Alignment Features

Features related to supplementary alignments show strong separation between classes. Chimeric reads frequently exhibit supplementary alignments, reflected by higher values of `has_sa`, `sa_count`, and `num_segments`, whereas clean reads consistently show a single alignment segment with no supplementary mappings. Table 4.1 shows that `has_sa` is present in chimeric reads (mean = 0.406) but absent in clean reads (mean = 0.000), while `num_segments` increases from a constant value of 1.000 in clean reads to a mean of 1.406 in chimeric reads. These patterns align with the expected structure of chimeric reads and indicate that alignment-based features are highly informative.

Clipping-Based Features

Clipping-related features, including `softclip_left`, `softclip_right`, and `total_clipped_bases`, display higher values and broader distributions in chimeric reads. In chimeric reads, `total_clipped_bases` reaches 25.44 on average, with a median of 19.0 and an IQR of 48.0, while `softclip_left` and `softclip_right` have averages of 12.55 and 12.90, medians of 0.0, and IQRs of 19.0. Clean reads maintain values near zero across all these metrics. These patterns indicate substantial clipping and increased variability in chimeric reads, reflecting junction-like alignment fragmentation, whereas clean reads remain unaltered.

K-mer Distribution Features

K-mer–based features, including `kmer_js_divergence` and `kmer_cosine_diff`, show only minor differences between clean and chimeric reads. In chimeric reads, `kmer_js_divergence` has a mean of 0.974 with a median of 0.986, and `kmer_cosine_diff` has a mean of 0.974 with a median of 0.986. Clean reads show similar values, with `kmer_js_divergence` at 0.976 with a median of 0.986, and `kmer_cosine_diff` at 0.976 with a median of 0.986. The close similarity of the means, medians, and overall ranges of values indicates that these features alone provide limited ability to distinguish clean from chimeric reads.

Microhomology Features

Microhomology-related features, including `microhomology_length` and `microhomology_gc`, exhibit nearly identical summary statistics between clean

and chimeric reads. Most reads in both classes have short or zero-length microhomologies. Table 4.1 shows that `microhomology_gc` has a mean of 0.172 and a median of 0.0 in both clean and chimeric reads, while `microhomology_length` averages 0.458 with a median of 0.0 in chimeric reads and 0.462 with a median of 0.0 in clean reads. These values indicate that microhomology features alone provide limited discriminatory power and are more appropriately considered as supporting evidence.

Overall, the summary statistics indicate that alignment-based and clipping-based features provide the strongest class separation, k-mer features contribute limited but complementary signal, and microhomology features exhibit minimal discriminative power on their own. These observations motivate the combined multi-feature approach used in subsequent modeling and evaluation.

Table 4.1: Summary statistics of selected key features by class.

Feature	Class	Mean	Std	Median	IQR
has_sa	chimeric	0.406	0.491	0.0	1.0
has_sa	clean	0.000	0.000	0.0	0.0
num_segments	chimeric	1.406	0.491	1.0	1.0
num_segments	clean	1.000	0.000	1.0	0.0
softclip_left	chimeric	12.55	21.90	0.0	19.0
softclip_left	clean	0.23	1.54	0.0	0.0
softclip_right	chimeric	12.90	22.12	0.0	19.0
softclip_right	clean	0.21	1.51	0.0	0.0
total_clipped_bases	chimeric	25.44	25.48	19.0	48.0
total_clipped_bases	clean	0.44	2.16	0.0	0.0
kmer_js_divergence	chimeric	0.974	0.025	0.986	0.043
kmer_js_divergence	clean	0.976	0.025	0.986	0.040
kmer_cosine_diff	chimeric	0.974	0.026	0.986	0.042
kmer_cosine_diff	clean	0.976	0.025	0.986	0.041
microhomology_length	chimeric	0.458	0.755	0.0	1.0
microhomology_length	clean	0.462	0.758	0.0	1.0
microhomology_gc	chimeric	0.172	0.361	0.0	0.0
microhomology_gc	clean	0.172	0.361	0.0	0.0

Boxplots were generated for each feature, with the x-axis representing the class (clean reads and chimeric reads) and the y-axis representing the feature value. Figure 4.1 presents a panel of selected key features, while boxplots for all numeric features are provided in Appendix B.

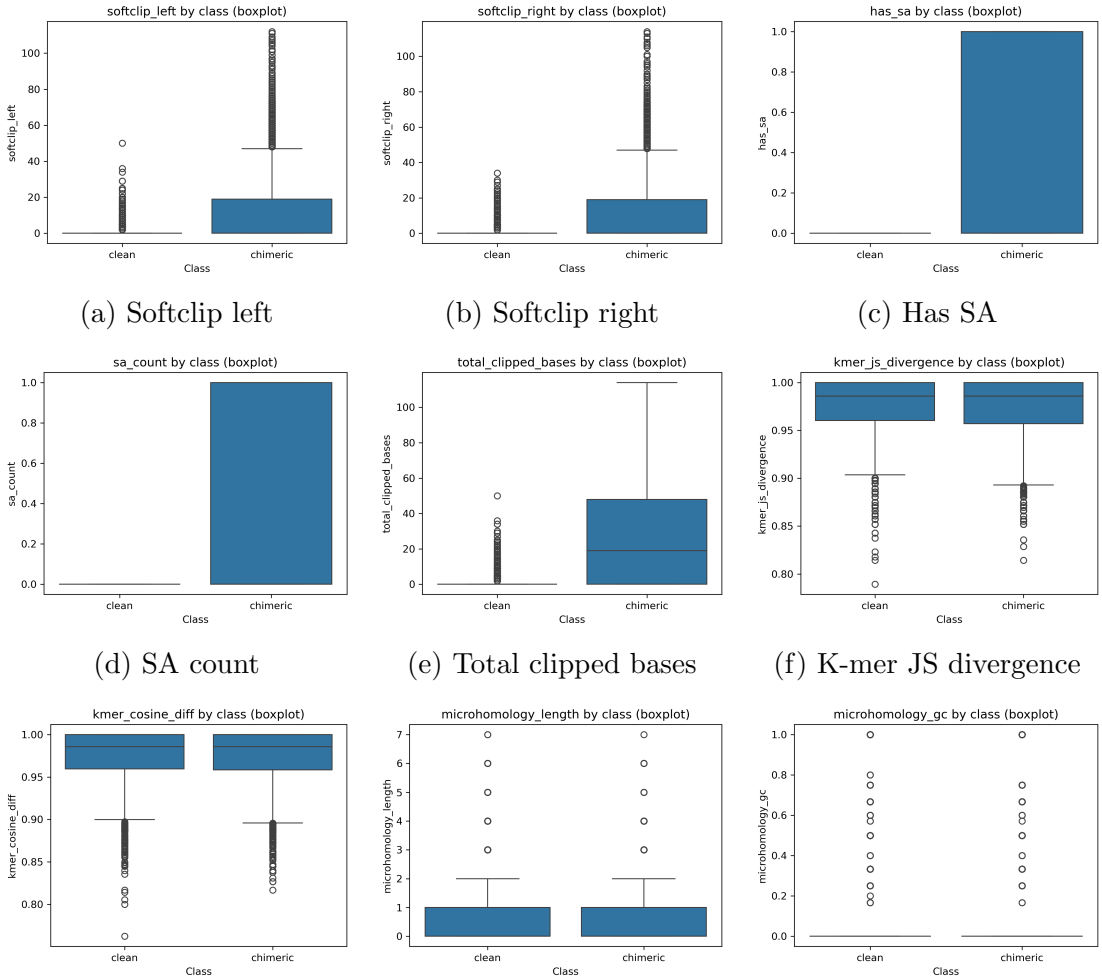
For clipping-related features (`softclip_left`, `softclip_right`, and `total_clipped_bases`), chimeric reads exhibit higher medians and longer upper whiskers than clean reads, indicating increased variability and the presence of split alignments.

Supplementary alignment features (`has_sa` and `sa_count`), show that clean reads are largely zero, whereas chimeric reads display a wider distribution, re-

flecting frequent supplementary alignments.

K-mer metrics (`kmer_js_divergence` and `kmer_cosine_diff`) show a slight upward shift for chimeric reads, but substantial overlap with clean reads indicates low discriminative power.

Microhomology features (`microhomology_length` and `microhomology_gc`) have nearly overlapping distributions for both classes, consistent with their low standalone predictive importance.



(a) Softclip left (b) Softclip right (c) Has SA

(d) SA count (e) Total clipped bases (f) K-mer JS divergence

(g) K-mer cosine difference (h) Microhomology length (i) Microhomology GC

Figure 4.1: Boxplots of selected features for clean and chimeric reads.

4.1.2 Correlation Analysis of Extracted Features

A feature correlation heatmap (Figure 4.2) was generated to examine relationships among the extracted variables and to identify patterns of redundancy and independence within the feature set. The analysis shows that alignment-related and clipping-related features form a strongly correlated cluster, including indicators of supplementary alignments, alignment segment counts, positional differences, and soft-clipping measures. These features capture related aspects of alignment fragmentation, which is a known characteristic of chimeric reads, and several show moderate correlations with the class label, supporting their relevance for distinguishing chimeric from clean reads. In contrast, general read-quality and alignment-quality metrics, such as read length, base quality, and mapping quality, exhibit weak correlations with most split-alignment features, indicating that they provide distinct information rather than overlapping with alignment-derived signals. Sequence-based features display a similar pattern of independence, as k-mer divergence metrics show weak correlations with other feature groups, while microhomology features exhibit generally low correlations with both alignment-based and k-mer-based features. Overall, the correlation structure highlights intentional redundancy within alignment-derived features and clear separation between feature families, supporting the use of features that capture different aspects of chimeric read characteristics to improve chimera classification.

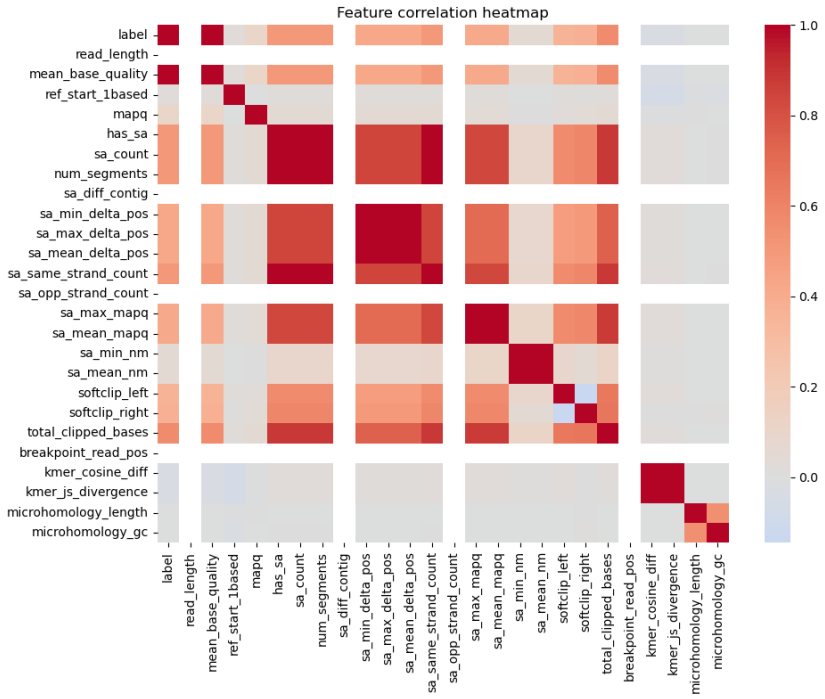


Figure 4.2: Feature correlation heatmap showing relationships among alignment-derived and sequence-derived features.

4.2 Baseline Classification Performance

Table 4.2 summarises the performance of thirteen baseline classifiers trained on the engineered feature set and evaluated on a held-out test set. All models were trained using default hyperparameters, without dedicated tuning.

The dummy baseline, which always predicts the same class regardless of the input features, achieved a test accuracy of approximately 0.50 and an F1-score of 0.67. This reflects the balanced class distribution and serves as a lower bound for meaningful model performance.

Across the remaining models, test F1-scores clustered within a narrow range,

from approximately 0.75 to 0.78, with ROC–AUC values between about 0.82 and 0.85. Ensemble methods, including gradient boosting, CatBoost, LightGBM, XGBoost, bagging trees, and random forest, exhibited very similar performance. Among these, CatBoost and gradient boosting achieved the highest scores, with test F1-scores of approximately 0.775 and ROC–AUC values of approximately 0.84. Linear models, namely logistic regression and the calibrated linear SVM, performed slightly worse, with test F1-scores around 0.75. In contrast, Gaussian Naive Bayes lagged behind with a substantially lower F1-score of approximately 0.66, despite exhibiting extremely high precision for the chimeric class.

Table 4.2: Performance of baseline classifiers on the held-out test set.

model	test_accuracy	test_precision	test_recall	test_f1	test_roc_auc
dummy_baseline	0.500188	0.500188	1.000000	0.666833	0.500000
logreg_l2	0.790797	0.945956	0.617000	0.746860	0.829807
linear_svm_calibrated	0.791422	0.947773	0.617000	0.747426	0.829602
random_forest	0.800050	0.910427	0.665750	0.769097	0.832766
extra_trees	0.797924	0.918833	0.653750	0.763950	0.826517
gradient_boosting	0.809053	0.947521	0.654500	0.774213	0.844844
xgboost	0.807303	0.942107	0.655000	0.772747	0.841042
lightgbm	0.806052	0.936231	0.657000	0.772146	0.841671
catboost	0.808803	0.941408	0.658750	0.775114	0.843362
knn	0.789671	0.902990	0.649250	0.755381	0.820898
gaussian_nb	0.745780	0.997975	0.492750	0.659749	0.826918
bagging_trees	0.800550	0.910830	0.666500	0.769742	0.837357
mlp	0.793047	0.949062	0.619500	0.749660	0.829611

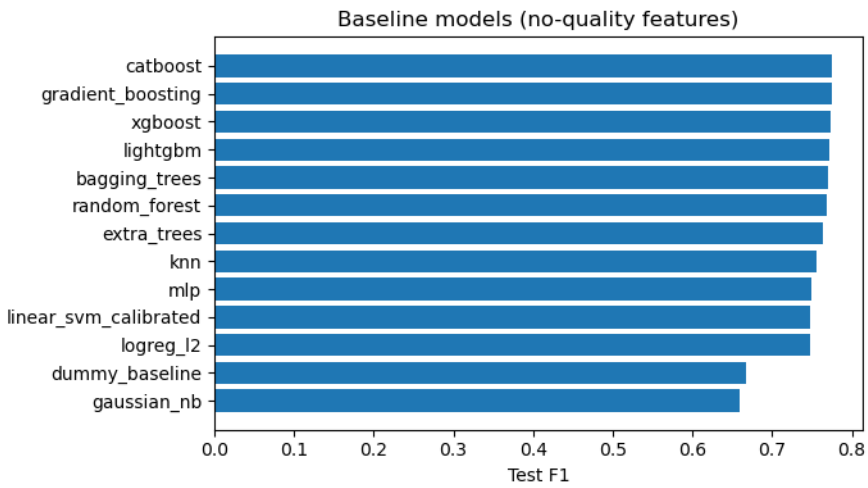


Figure 4.3: Test F1 of all baseline classifiers.

4.3 Effect of Hyperparameter Tuning

To assess whether performance could be improved further, ten model families underwent randomised hyperparameter search. The tuned metrics are summarised in Table 4.3. Overall, tuning yielded modest but consistent gains for tree-based ensembles and boosting methods, while leaving linear models essentially unchanged or slightly worse.

CatBoost, gradient boosting, LightGBM, random forest, bagging trees, and extra trees experienced small increases in test F1 after tuning, typically on the order of ΔF1 $\approx$ 0.002–0.006, with corresponding improvements in ROC–AUC of up to approximately ΔAUC $\approx$ 0.009. In contrast, XGBoost and the multilayer perceptron showed negligible change or slight decreases in F1, while linear models did not benefit from tuning.

After tuning, gradient boosting achieved the best overall test performance,

with a test F1-score of 0.776 and a ROC–AUC of 0.846. LightGBM and bagging trees followed closely, attaining test F1-scores of 0.774 and 0.772 and ROC–AUC values of 0.843 and 0.842, respectively. Random forest also improved modestly to a test F1-score of 0.772 with a ROC–AUC of 0.839. CatBoost, with a test F1-score of 0.775 and ROC–AUC of 0.843, achieved marginal changes relative to its baseline performance.

Table 4.3: Performance of tuned classifiers on the held-out test set.

model_name	test_f1_base	test_roc_auc_base	test_f1_tuned	test_roc_auc_tuned
gradient_boosting	0.774213	0.844844	0.776460	0.845858
catboost	0.775114	0.843362	0.775289	0.842918
lightgbm	0.772146	0.841671	0.773802	0.843451
bagging_trees	0.769742	0.837357	0.772422	0.841870
random_forest	0.769097	0.832766	0.772376	0.838799
xgboost	0.772747	0.841042	0.770118	0.843225
extra_trees	0.763950	0.826517	0.769878	0.834912
mlp	0.749660	0.829611	0.749167	0.828506
logreg_l2	0.746860	0.829807	0.745187	0.825632
linear_svm_calibrated	0.747426	0.829602	0.744848	0.825147

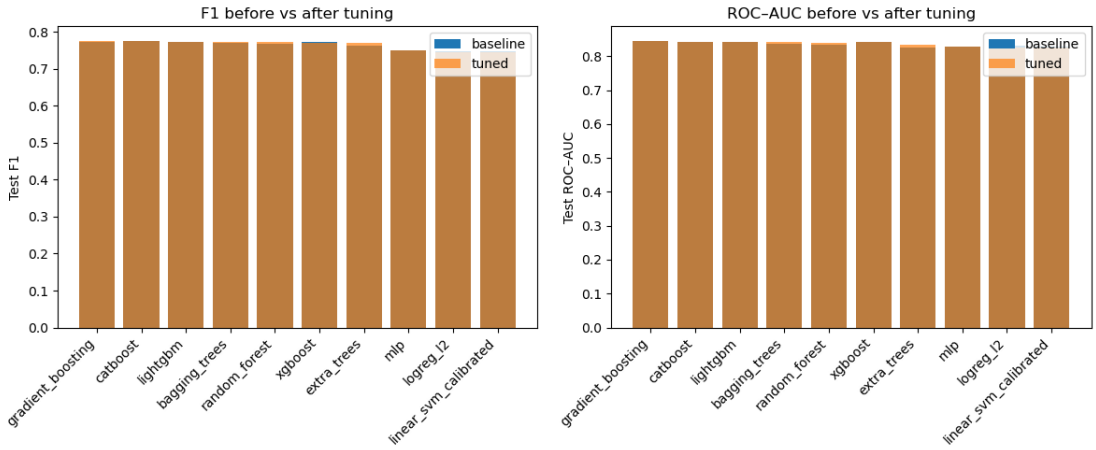


Figure 4.4: Comparison of test F1 (left) and ROC–AUC (right) for baseline and tuned models.

Because improvements are small and within cross-validation variability, tun-

ing was interpreted as stabilising and slightly refining the models rather than completely altering their behaviour or their relative ranking.

4.3.1 Confusion Matrix and Error Patterns

Gradient Boosting was selected as the final classifier, as it demonstrated the best overall performance. Its classification report and confusion matrix show an overall accuracy of approximately 0.81, with a macro-averaged F1-score of about 0.80. For clean reads, the model achieved precision of 0.74 and recall of 0.96, while for chimeric reads it achieved precision of 0.94 and recall of 0.66, corresponding to an F1-score of 0.777 for the chimeric class.

Errors were asymmetric, with false negatives (chimeric reads predicted as clean) occurring more often than false positives. Specifically, the model misclassified 1,352 chimeric reads as clean and 181 clean reads as chimeric. This pattern indicates that the classifier is conservative, prioritising the avoidance of false chimera calls even if some true chimeric reads are missed. Consultation with PGC Visayas suggested that this conservative behaviour is generally acceptable, although further assessment is still needed to determine its downstream impact on analysis workflows.

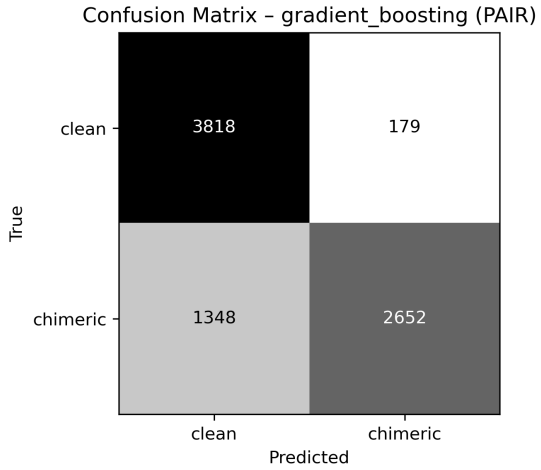


Figure 4.5: Confusion matrix of the tuned Gradient Boosting classifier on the held-out test set.

4.3.2 ROC and Precision–Recall Curves

The receiver operating characteristic (ROC) and precision–recall (PR) curves further demonstrate the strong performance of the tuned Gradient Boosting classifier. The model achieved a ROC–AUC of approximately 0.84 and an average precision of approximately 0.88 on the held-out test set.

The PR curve shows that precision remains above 0.9 across a broad range of recall values up to roughly 0.5–0.6, after which precision gradually declines. This behaviour suggests that the classifier can assign very high confidence to a subset of chimeric reads, while the recovery of more ambiguous reads requires accepting lower precision. Overall, these curves indicate that Gradient Boosting provides a strong balance between sensitivity and specificity for the detection of chimeric reads.

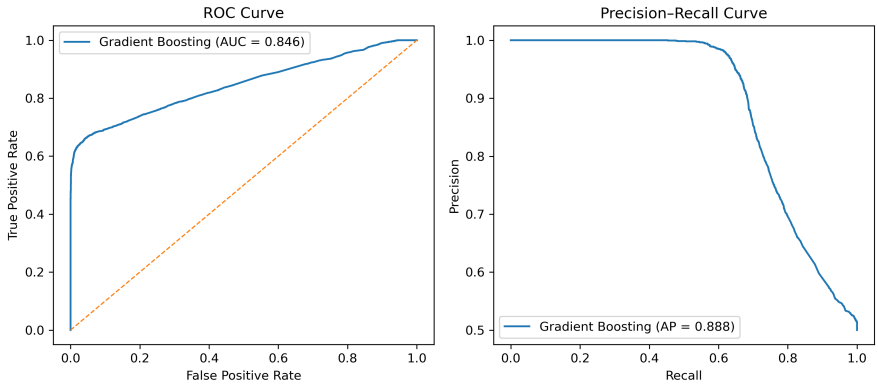


Figure 4.6: ROC and precision–recall curves for the tuned Gradient Boosting classifier on the held-out test set.

4.4 Feature Importance

4.4.1 Permutation Importance of Individual Features

To understand how the final classifier made predictions, feature importance was quantified using permutation importance for the tuned Gradient Boosting model.

As shown in Figure 4.7, `total_clipped_bases` provides the strongest predictive signal, with an importance of 0.117. This is followed by `kmer_js_divergence` and `kmer_cosine_diff`, indicating that both clipping-based alignment irregularities and k-mer compositional shifts are central to the model's decision-making process. In contrast, soft-clipping features contribute less strongly, while microhomology-related features such as `microhomology_length` and `microhomology_gc` provide minimal predictive value.

These results indicate that accurate detection of chimeric reads relies primarily on signals associated with breakpoint-related clipping patterns and abrupt com-

positional changes in sequence structure. By contrast, explicit microhomology features appear to add little information for this dataset.

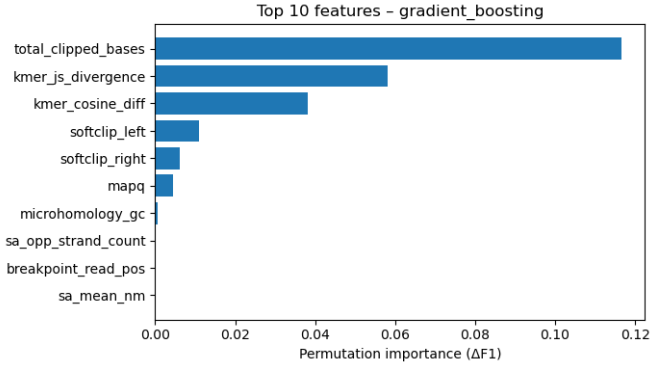


Figure 4.7: Permutation-based feature importance for the tuned Gradient Boosting classifier.

4.4.2 Feature Family Importance

To evaluate broader predictive patterns, features were grouped into five families: SA_structure, Clipping, Kmer_jump, Micro_homology, and Other.

For the tuned Gradient Boosting model, the Clipping family contributed the largest cumulative importance (0.134), followed by Kmer_jump (0.096). The remaining families, namely SA_structure, Micro_homology, and Other, contributed only minimally. This pattern shows that the model draws most of its predictive power from alignment-based clipping signals and k-mer compositional changes, while explicit microhomology descriptors and other auxiliary alignment metrics play only a minor role.

Taken together, both feature-level and feature-family analyses indicate that Gradient Boosting relies primarily on breakpoint-associated clipping irregularities

and shifts in local sequence composition to distinguish chimeric from clean reads.

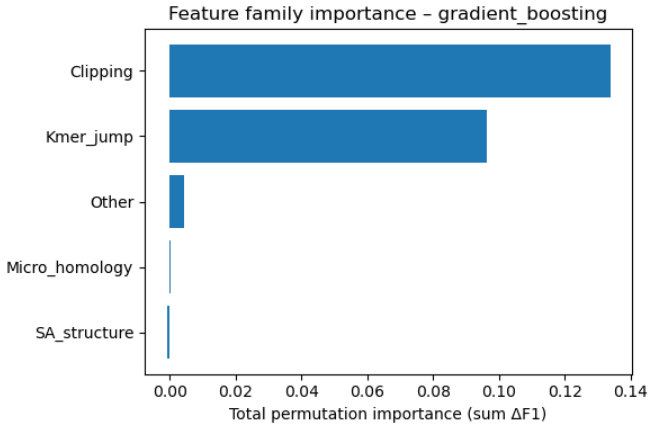


Figure 4.8: Aggregated feature family importance for the tuned Gradient Boosting classifier.

4.5 Feature Selection

Feature selection was performed to identify the smallest subset reaching 95% cumulative importance. Three configurations were evaluated as references: the full Gradient Boosting model with all 24 features, a reduced model with the top-k features, and an ablation model excluding microhomology features, using the tuned Gradient Boosting classifier to assess feature contributions and overall classification performance.

4.5.1 Cumulative Importance Curve

The cumulative importance curve was computed using the tuned Gradient Boosting classifier. Figure 4.9 shows that the importance rises steeply for the top-ranked features and then gradually plateaus, indicating that a relatively small number

of predictors account for most of the model's performance. A cumulative importance threshold of 95% was reached at $k = 4$ features: `total_clipped_bases`, `kmer_js_divergence`, `kmer_cosine_diff`, and `softclip_left`.

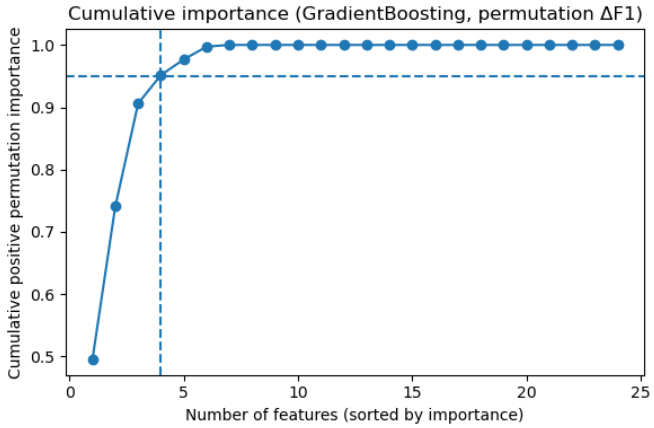


Figure 4.9: Cumulative importance curve of features sorted by importance for the tuned Gradient Boosting classifier.

4.5.2 Performance Comparison Across Feature Sets

Classification performance was compared across three feature sets using the tuned Gradient Boosting classifier. The full model, incorporating all 24 engineered features, achieved an F1-score of 0.7765 and a ROC–AUC of 0.8459. A reduced model using only the top four features (`total_clipped_bases`, `kmer_js_divergence`, `kmer_cosine_diff`, and `softclip_left`) achieved nearly equivalent performance, with an F1-score of 0.7768 and a ROC–AUC of 0.8369. An ablation model excluding microhomology features (`microhomology_length` and `microhomology_gc`) also performed comparably, with an F1-score of 0.7761 and a ROC–AUC of 0.8444.

These findings indicate that clipping and k-mer features capture nearly all of the predictive signal used by the model, while microhomology features are largely

redundant in this dataset.

Table 4.4: Test set performance of feature set variants using the tuned Gradient Boosting classifier.

Variant	No. of Features	Test F1	ROC–AUC
Full Gradient Boost	24	0.7765	0.8459
Selected (top-4)	4	0.7768	0.8369
No microhomology	22	0.7761	0.8444

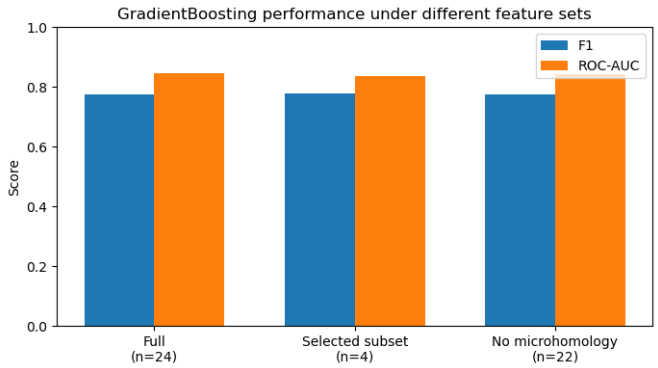


Figure 4.10: Comparison of F1 and ROC–AUC across feature set variants for the tuned Gradient Boosting classifier.

4.5.3 Interpretation and Final Feature Set Choice

The full 24-feature Gradient Boosting model was retained as the primary configuration for the remainder of the study, while the four-feature subset is presented as a lightweight alternative. Clipping features reflect alignment junctions and mapping disruptions typical of chimeric reads, whereas k-mer divergence features capture abrupt changes in sequence composition across breakpoints. Microhomology features appear largely redundant, suggesting that their signal is either indirectly captured by clipping and k-mer descriptors or not strongly expressed in the simulation dataset.

4.6 Convolutional Neural Network (CNN) Performance and Classification Results

The CNN achieved strong and reasonably consistent performance across the five cross-validation folds, as summarized in Table 4.5. The mean validation loss across folds was 0.4444. The corresponding mean classification metrics were 0.8664 for accuracy, 0.8214 for precision, 0.9374 for recall, 0.8754 for F1-score, and 0.9359 for ROC–AUC. Across folds, F1-scores ranged from 0.8637 to 0.8898, while ROC–AUC values ranged from 0.9272 to 0.9467, indicating only modest variation in performance between validation splits.

Table 4.5: Five-fold cross-validation performance of the CNN model.

Metric	Mean	Std. Dev.
Loss	0.4444	0.0308
Accuracy	0.8664	0.0139
Precision	0.8214	0.0226
Recall	0.9374	0.0083
F1-score	0.8754	0.0110
ROC–AUC	0.9359	0.0081

Figure 4.11 illustrates the learning behaviour of the model across epochs. Training loss decreases steadily, while validation loss declines during the early epochs and then stabilizes with only small fluctuations. Validation F1 follows a similar pattern, rising early in training before reaching a plateau, which suggests that the network learns discriminative patterns quickly and then converges to a stable performance region.

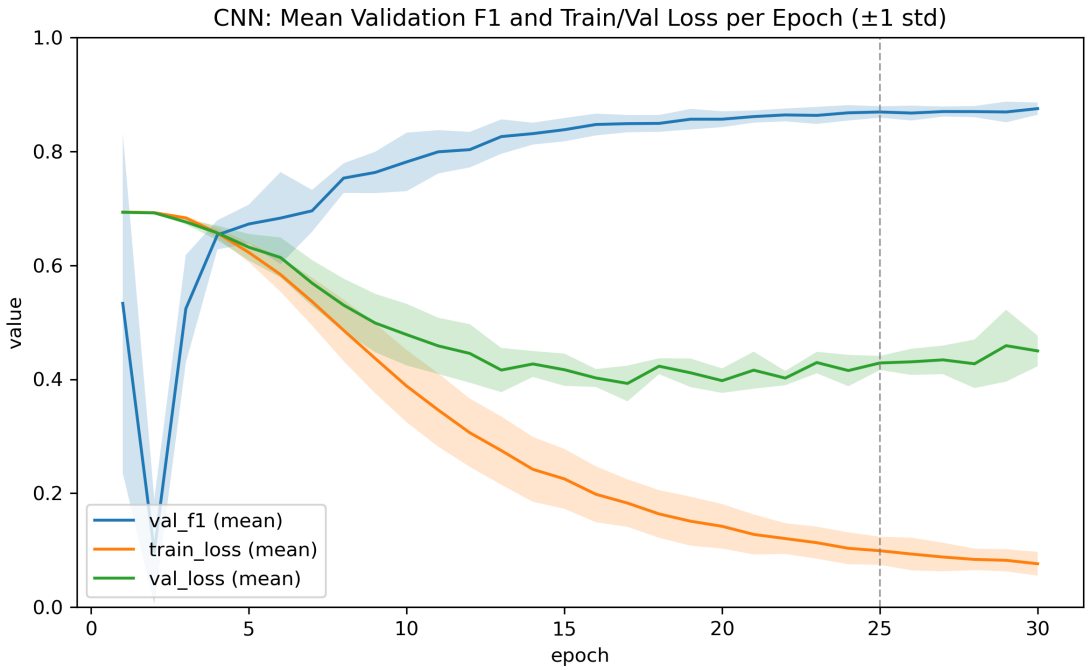


Figure 4.11: CNN training and validation loss and validation F1 across epochs.

When evaluated on the held-out balanced test set, the final CNN achieved a test loss of 0.3034, accuracy of 0.9092, precision of 0.8724, recall of 0.9588, F1-score of 0.9135, and ROC–AUC of 0.9607, as shown in Table 4.6. These results indicate that the model generalizes well beyond the validation folds and maintains strong classification performance on previously unseen data.

Table 4.6: Held-out test performance of the CNN model.

Metric	Value
Loss	0.3034
Accuracy	0.9092
Precision	0.8724
Recall	0.9588
F1-score	0.9135
ROC–AUC	0.9607

As shown in Figure 4.12, the held-out test results indicate that the CNN is particularly effective at recovering chimeric reads, as reflected by its high recall. At the same time, its precision is lower than its recall, indicating that the model prioritizes sensitivity and accepts a moderate number of false-positive predictions in return. Nevertheless, the combination of a high F1-score and a ROC–AUC above 0.96 shows that the classifier still provides strong overall discrimination between clean and chimeric reads.

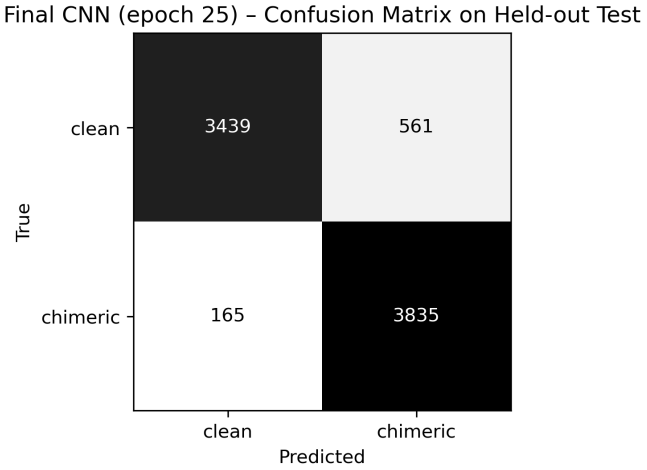


Figure 4.12: Confusion matrix of the CNN model on the held-out test set.

4.7 Recurrent Neural Network (RNN) Performance and Classification Results

Across five cross-validation folds, the RNN-kmer BiGRU model achieved consistently high performance with low variability, as shown in Table 4.7. The mean cross-validation accuracy was 0.9303, mean precision was 0.9137, mean recall was 0.9505, mean F1-score was 0.9317, and mean ROC–AUC was 0.9734. Mean vali-

dation loss across folds was 0.3397. Individual fold F1-scores ranged from 0.9252 to 0.9386, while ROC–AUC ranged from 0.9672 to 0.9780, indicating only small variation across folds.

Table 4.7: Five-fold cross-validation performance of the RNN-kmer BiGRU model.

Metric	Mean	Std. Dev.
Loss	0.3397	0.0619
Accuracy	0.9303	0.0055
Precision	0.9137	0.0118
Recall	0.9505	0.0094
F1-score	0.9317	0.0051
ROC–AUC	0.9734	0.0048

The training curves also show stable optimization behavior. As shown in Figure 4.13, training loss decreases rapidly during the early epochs and continues to decline toward near-zero values, while validation loss drops sharply in the first several epochs before plateauing and fluctuating mildly. At the same time, validation F1 and validation AUC increase quickly and stabilize at high values, reaching approximately 0.93 and 0.97, respectively.

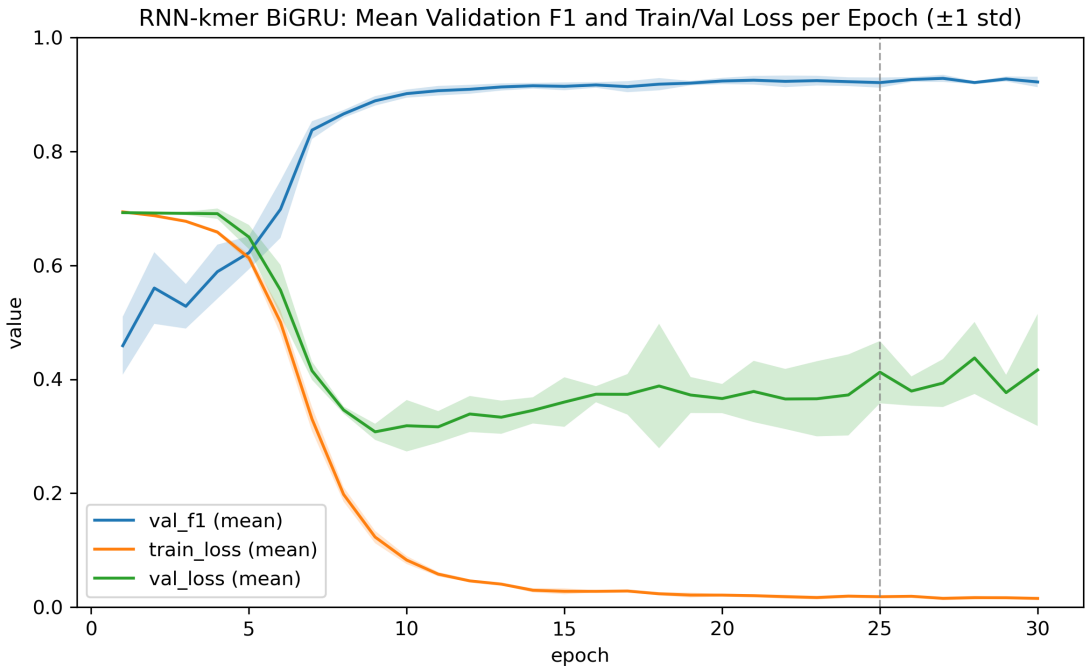


Figure 4.13: RNN-kmer BiGRU training and validation loss and validation F1 across epochs.

On the held-out balanced test set ($n = 8000$; 4000 clean and 4000 chimeric reads), the final RNN-kmer BiGRU model achieved a test loss of 0.2432, accuracy of 0.9516, precision of 0.9394, recall of 0.9655, F1-score of 0.9523, and ROC–AUC of 0.9849, as summarized in Table 4.8. These values exceed those of both the tuned Gradient Boosting classifier and the CNN.

Table 4.8: Held-out test performance of the RNN-kmer BiGRU model.

Metric	Value
Loss	0.2432
Accuracy	0.9516
Precision	0.9394
Recall	0.9655
F1-score	0.9523
ROC–AUC	0.9849

Figure 4.14 shows that the model correctly classified 3751 of 4000 clean reads and 3862 of 4000 chimeric reads. Only 249 clean reads were incorrectly labeled as chimeric, while 138 chimeric reads were misclassified as clean.

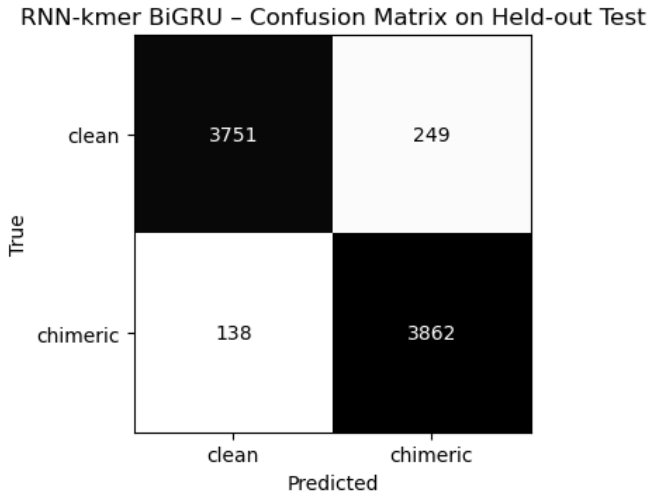


Figure 4.14: Confusion matrix of the RNN-kmer BiGRU model on the held-out test set.

4.7.1 External Assembly Testing

3200 Paired-end Reads

At the minimum read depth (3200), assembly outcomes depended strongly on how many reads were retained after filtering. At 5% chimeras, all filters resolved minor branching by reducing the graph from 2 nodes (unfiltered) to 1 node. However, at higher chimera rates, filtering effects diverged. The CNN filter produced fragmentation at 15–50% chimeras (2 nodes), consistent with aggressive read removal, where CNN retained only 48.78–75.19% of reads. BiGRU maintained a 1-node graph at 10–15% chimeras while retaining 75.56–78.94% of reads, but at 50%

chimeras the graph fragmented into 8 nodes after retention dropped to 44.59%. In contrast, GB maintained a 1-node graph across these conditions by retaining more reads (77.88–95.22%).

10K Paired-end Reads

At 10K reads and 5–15% chimeras, all assemblies produced a clean 1-node graph, indicating that filtering mainly affected read retention rather than assembly structure. At 50% chimeras, the unfiltered graph branched into 3 nodes. Filtering reduced this complexity, with GB, CNN, and BiGRU each producing 2 nodes. GB retained 78.94% of reads, CNN retained 49.47%, and BiGRU retained 44.68%. The residual nodes under all three filters suggest that even after filtering, some ambiguous reads remained that were sufficient to sustain minor branches, although graph complexity was still reduced relative to the unfiltered condition.

20K Paired-end Reads

At 20K reads, unfiltered assemblies showed increased branching at 10–15% chimeras (3 nodes) and extreme fragmentation at 50% (966 nodes). All filters restored contiguity. BiGRU produced a 1-node graph at 5–15% chimeras while retaining 75.13–83.84% of reads, and reduced the 50% chimera graph to 2 nodes with 44.85% retention. At this highest chimera level, CNN produced the simplest graph (1 node) with 49.89% retention, whereas GB produced 3 nodes while retaining substantially more reads (78.49%). These results indicate that at higher coverage, more stringent filtering reduced graph complexity relative to GB. In the 50% chimera condition, CNN yielded a more simplified graph than BiGRU,

as BiGRU still produced 2 nodes despite retaining a smaller proportion of reads.

Across read depths, GB consistently retained the highest proportion of reads and generally preserved contiguous assemblies, making it the most reliable at low depth (3200). BiGRU showed an intermediate behaviour between GB and CNN. It retained fewer reads than GB and, under high chimera burden, reduced graph complexity more than GB, although not as effectively as CNN. At the minimum depth, BiGRU became risky at high chimera percentage, where retention dropped to 44.59% and the assembly fragmented into 8 nodes. Overall, assembly outcomes depended on both read depth and chimera percentage. At higher coverage, stronger filtering could reduce graph complexity, but the extent of simplification varied across models: CNN achieved the strongest simplification at high chimera percentage, whereas BiGRU showed only partial simplification under the same condition. At minimum coverage, stronger filtering could reduce read support sufficiently to split the assembly into multiple nodes. In some unfiltered conditions, Bandage still showed a single-node graph. This does not imply the absence of chimeric reads; rather, SPAdes outputs a simplified assembly graph after trimming tips and removing erroneous, including chimeric, connections. If chimeric reads do not generate an alternative path with sufficient support to persist through simplification, the assembly collapses to one dominant segment and appears as a single node in Bandage.

///insert cite Bankevich2012SPAdes,Antipov2016SPAdesHybrid,Wick2015Bandage

4.7.2 Comparison Across Models

Direct comparison of the three best-performing approaches, shown in Table 4.9 and Figure 4.15, shows a clear ranking. The tuned Gradient Boosting model achieved an F1-score of 0.7765 and ROC–AUC of 0.8459, while the CNN improved substantially to an F1-score of 0.9135 and ROC–AUC of 0.9607. The RNN-kmer BiGRU surpassed both, reaching an F1-score of 0.9523 and ROC–AUC of 0.9849.

Table 4.9: Comparison of held-out test performance across the top-performing models.

Model	Accuracy	Precision	Recall	F1-score	ROC–AUC
Gradient Boosting (tuned)	0.8091	0.9368	0.6630	0.7765	0.8459
CNN1D	0.9093	0.8724	0.9588	0.9135	0.9607
RNN-kmer BiGRU	0.9516	0.9394	0.9655	0.9523	0.9849

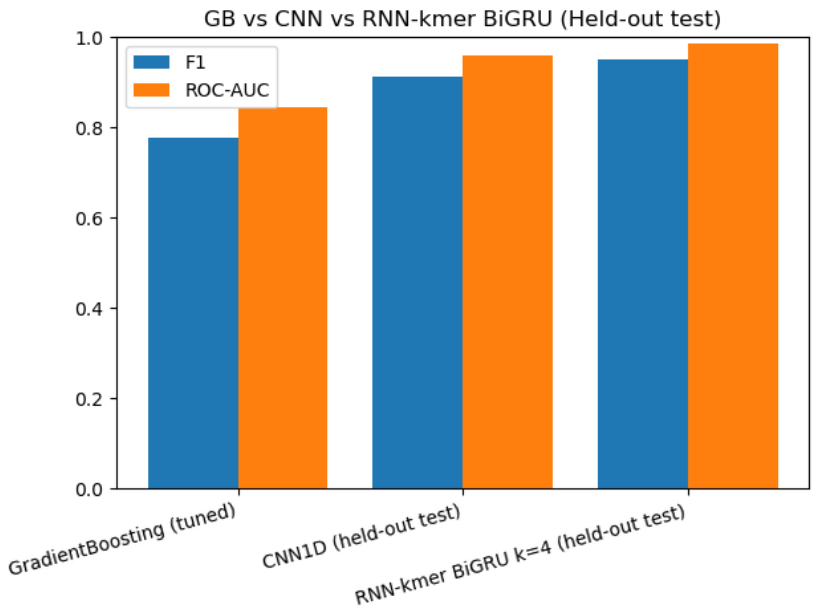


Figure 4.15: Comparison of held-out test F1 and ROC–AUC for Gradient Boosting, CNN, and RNN models.

4.8 Summary of Findings

///NOTE: TO BE REVISED AFTER FINAL CHECKS ON THE FULL RESULTS

All evaluated machine learning and deep learning models substantially outperformed the dummy baseline, demonstrating that both the engineered feature set and sequence-based representations contain meaningful signals for detecting PCR-induced chimeric reads. Among the feature-based classifiers, the best-performing classical models achieved test F1-scores of approximately 0.77 and ROC–AUC values around 0.84 on held-out simulated mitochondrial reads, indicating reliable discrimination between clean and chimeric sequences. Tree-based ensemble and boosting methods consistently showed the strongest performance among these classical approaches, with CatBoost and Gradient Boosting ranking among the top models before and after hyperparameter tuning.

Analysis of feature behaviour revealed clear differences in how effectively feature groups distinguished clean and chimeric reads. Alignment- and clipping-based features, such as soft-clipping measures and total clipped bases, showed strong separation between clean and chimeric reads and emerged as the most informative signals. K-mer divergence features provided additional but weaker separation, contributing complementary information beyond alignment irregularities. In contrast, microhomology features and several supplementary alignment (SA) structure metrics exhibited minimal class separation and contributed little to overall predictive performance.

Feature selection results further supported these observations. A reduced sub-

set of four features, dominated by clipping-based and k-mer divergence metrics, achieved nearly identical performance to the full 23-feature model. In addition, removing explicit microhomology features did not substantially reduce classification performance, indicating that these features contributed little under the simulated conditions used in this study.

The deep learning models achieved substantially stronger performance than the feature-based classifiers. The one-dimensional CNN improved held-out test performance to an F1-score of 0.9135 and a ROC–AUC of 0.9607, showing that sequence-based modeling can capture discriminative patterns beyond those represented by the engineered features alone. The RNN-kmer BiGRU model achieved the best overall performance among all evaluated methods, with a held-out test accuracy of 0.9516, precision of 0.9394, recall of 0.9655, F1-score of 0.9523, and ROC–AUC of 0.9849. Cross-validation results for the RNN were also consistent across folds, with mean F1-score of 0.9317 and mean ROC–AUC of 0.9734.

Overall, these findings show that PCR-induced chimeric reads in simulated mitochondrial data can be detected effectively using both engineered-feature classifiers and sequence-based neural models, but the highest classification performance was achieved by the RNN. While alignment-based and k-mer-based engineered features provide a compact and interpretable basis for detection, the deep learning results indicate that sequential k-mer representations are highly effective for this objective. Taken together, the results support the use of machine learning as a pre-assembly chimera detection step, with the RNN-kmer BiGRU providing the strongest overall predictive performance in this study.

Appendix A

Complete Per-Class Summary Statistics

Table A.1: Complete per-class summary statistics for all extracted features.

Feature	Class	Mean	Std	Median	Q1	Q3	IQR	Min	Max	n
breakpoint_read_pos	chimeric	75.000	0.000	75.000	75.000	75.000	0.000	75.000	75.000	20000
breakpoint_read_pos	clean	75.000	0.000	75.000	75.000	75.000	0.000	75.000	75.000	19983
has_sa	chimeric	0.406	0.491	0.000	0.000	1.000	1.000	0.000	1.000	20000
has_sa	clean	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	19983
kmer_cosine_diff	chimeric	0.974	0.026	0.986	0.958	1.000	0.042	0.817	1.000	20000
kmer_cosine_diff	clean	0.976	0.025	0.986	0.959	1.000	0.041	0.814	1.000	19983
kmer_js_divergence	chimeric	0.974	0.025	0.986	0.957	1.000	0.043	0.811	1.000	20000
kmer_js_divergence	clean	0.976	0.025	0.986	0.959	1.000	0.040	0.817	1.000	19983
mapq	chimeric	59.987	0.355	60.000	60.000	60.000	0.000	43.000	60.000	20000
mapq	clean	59.663	2.036	60.000	60.000	60.000	0.000	0.000	60.000	19983
mean_base_quality	chimeric	40.000	0.000	40.000	40.000	40.000	0.000	40.000	40.000	20000
mean_base_quality	clean	13.000	0.000	13.000	13.000	13.000	0.000	13.000	13.000	19983
microhomology_gc	chimeric	0.172	0.361	0.000	0.000	0.000	0.000	0.000	1.000	20000
microhomology_gc	clean	0.172	0.361	0.000	0.000	0.000	0.000	0.000	1.000	19983
microhomology_length	chimeric	0.458	0.755	0.000	0.000	1.000	1.000	0.000	5.000	20000
microhomology_length	clean	0.462	0.758	0.000	0.000	1.000	1.000	0.000	5.000	19983

Continued on next page

Feature	Class	Mean	Std	Median	Q1	Q3	IQR	Min	Max	n
num_segments	chimeric	1.406	0.491	1.000	1.000	2.000	1.000	1.000	2.000	20000
num_segments	clean	1.000	0.000	1.000	1.000	1.000	0.000	1.000	1.000	19983
read_length	chimeric	150.000	0.000	150.000	150.000	150.000	0.000	150.000	150.000	20000
read_length	clean	150.000	0.000	150.000	150.000	150.000	0.000	150.000	150.000	19983
ref_start_1based	chimeric	8428.635	4248.348	8433.000	5013.000	11786.250	6773.250	1.000	16521.000	20000
ref_start_1based	clean	8200.121	4626.918	8240.000	3639.000	11565.000	7926.000	1.000	16521.000	19983
sa_count	chimeric	0.406	0.491	0.000	0.000	1.000	1.000	0.000	1.000	20000
sa_count	clean	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	19983
sa_diff_contig	chimeric	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	20000
sa_diff_contig	clean	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	19983
sa_max_delta_pos	chimeric	1573.531	2364.996	0.000	0.000	2826.250	2826.250	0.000	16519.000	20000
sa_max_delta_pos	clean	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	19983
sa_max_mapq	chimeric	14.104	21.424	0.000	0.000	27.000	27.000	0.000	60.000	20000
sa_max_mapq	clean	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	19983
sa_mean_delta_pos	chimeric	1573.531	2364.996	0.000	0.000	2826.250	2826.250	0.000	16519.000	20000
sa_mean_delta_pos	clean	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	19983
sa_mean_mapq	chimeric	14.104	21.424	0.000	0.000	27.000	27.000	0.000	60.000	20000
sa_mean_mapq	clean	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	19983

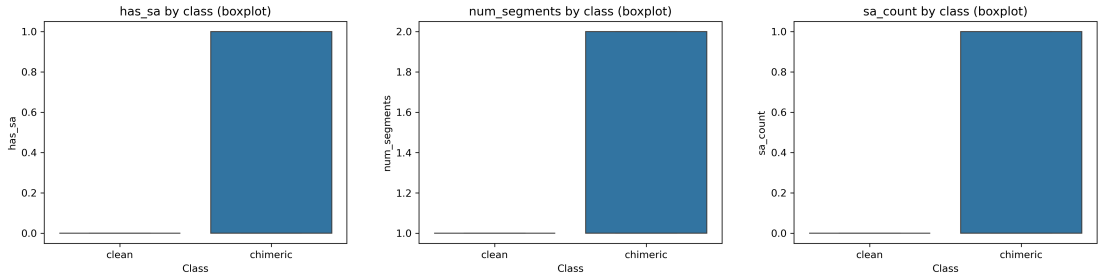
Continued on next page

Feature	Class	Mean	Std	Median	Q1	Q3	IQR	Min	Max	n
sa_mean_nm	chimeric	0.022	0.319	0.000	0.000	0.000	0.000	0.000	6.000	20000
sa_mean_nm	clean	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	19983
sa_min_delta_pos	chimeric	1573.531	2364.996	0.000	0.000	2826.250	2826.250	0.000	16519.000	20000
sa_min_delta_pos	clean	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	19983
sa_min_nm	chimeric	0.022	0.319	0.000	0.000	0.000	0.000	0.000	6.000	20000
sa_min_nm	clean	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	19983
sa_opp_strand_count	chimeric	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	20000
sa_opp_strand_count	clean	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	19983
sa_same_strand_count	chimeric	0.406	0.491	0.000	0.000	1.000	1.000	0.000	1.000	20000
sa_same_strand_count	clean	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	19983
softclip_left	chimeric	12.546	21.898	0.000	0.000	19.000	19.000	0.000	150.000	20000
softclip_left	clean	0.225	1.543	0.000	0.000	0.000	0.000	0.000	56.000	19983
softclip_right	chimeric	12.896	22.123	0.000	0.000	19.000	19.000	0.000	150.000	20000
softclip_right	clean	0.212	1.513	0.000	0.000	0.000	0.000	0.000	55.000	19983
total_clipped_bases	chimeric	25.442	25.481	19.000	0.000	48.000	48.000	0.000	150.000	20000
total_clipped_bases	clean	0.437	2.157	0.000	0.000	0.000	0.000	0.000	110.000	19983

Appendix B

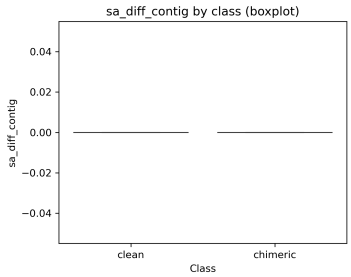
Boxplots for All Numeric Features by Feature Family

B.0.1 SA Structure (Supplementary Alignment and Segment Metrics)

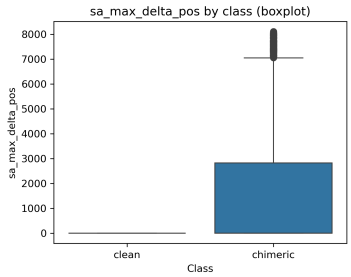


(a) Has SA (b) Number of segments (c) SA count

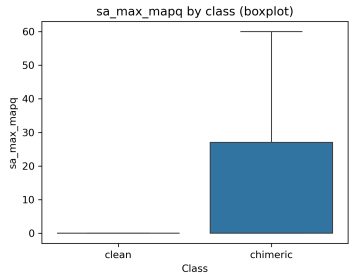
Figure B.1: Boxplots of SA Structure features by class (1/2).



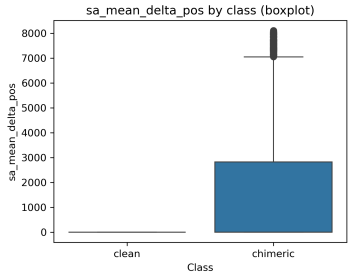
(a) SA different contig



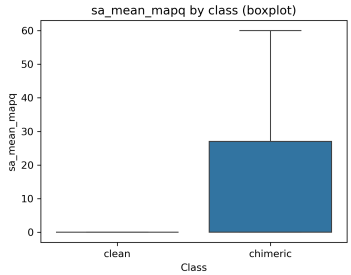
(b) SA max Δ position



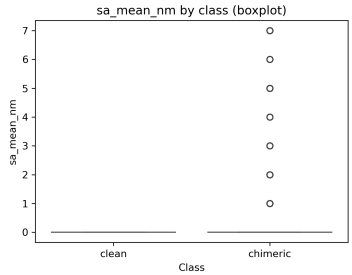
(c) SA max MAPQ



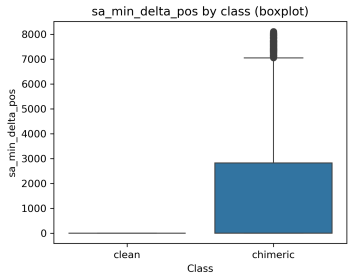
(d) SA mean Δ position



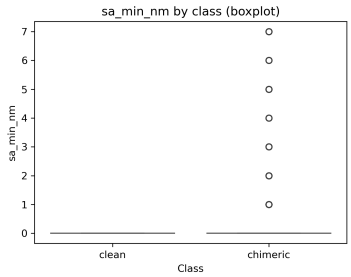
(e) SA mean MAPQ



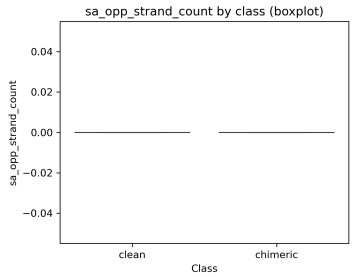
(f) SA mean NM



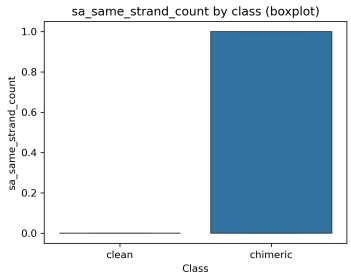
(g) SA min Δ position



(h) SA min NM



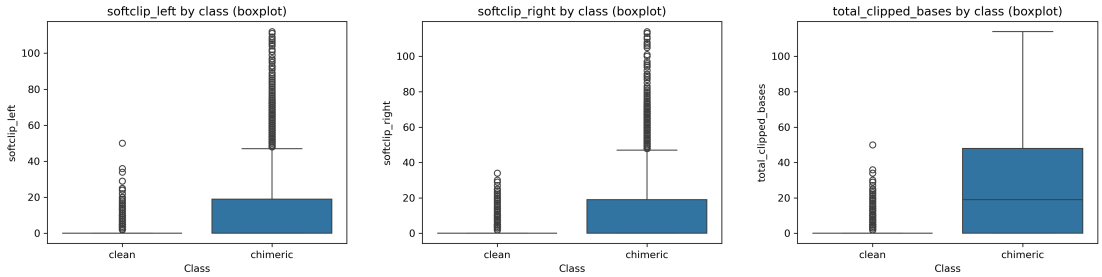
(i) SA opposite strand count



(j) SA same strand count

Figure B.2: Boxplots of SA Structure features by class (2/2).

B.0.2 Clipping-Based Features



(a) Softclip left (b) Softclip right (c) Total clipped bases

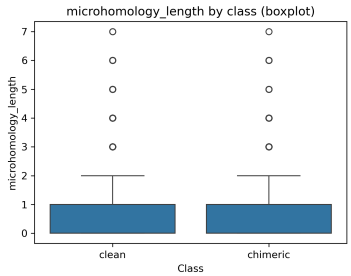
Figure B.3: Boxplots of clipping-based features by class.

B.0.3 K-mer Features

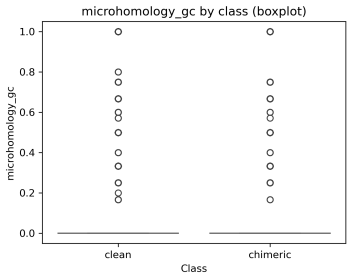
(a) K-mer cosine difference (b) K-mer JS divergence

Figure B.4: Boxplots of k-mer features by class.

B.0.4 Microhomology Features



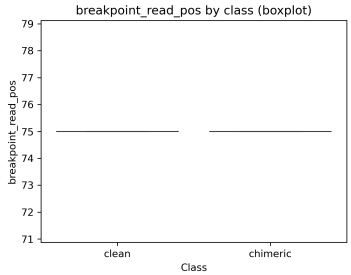
(a) Microhomology length



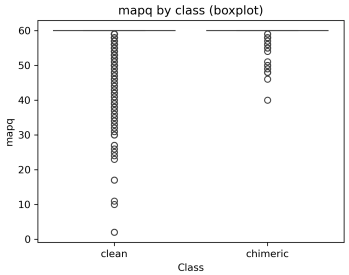
(b) Microhomology GC

Figure B.5: Boxplots of microhomology features by class.

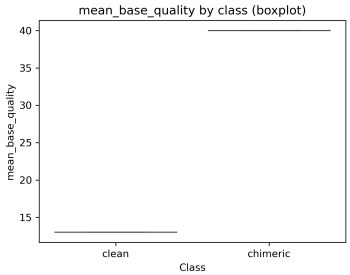
B.0.5 Others



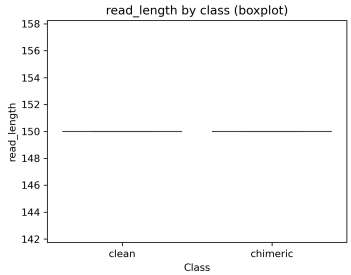
(a) Breakpoint read position



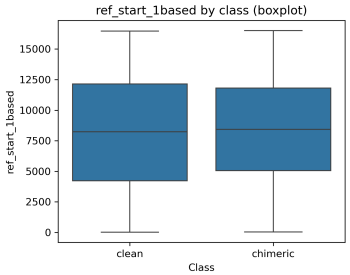
(b) MAPQ



(c) Mean base quality



(d) Read length



(e) Reference start (1-based)

Figure B.6: Boxplots of other numeric features by class.

References

Anderson, S., Bankier, A., Barrell, B., Bruijn, M., Coulson, A., Drouin, J., ... Young, I. (1981, 04). Sequence and organization of the human mitochondrial genome. *Nature*, *290*, 457-465. doi: 10.1038/290457a0

Arango, G., Garner, E., Pruden, A., Heath, L., Vikesland, P., & Zhang, L. (2018, 02). Deeparg: A deep learning approach for predicting antibiotic resistance genes from metagenomic data. *Microbiome*, *6*. doi: 10.1186/s40168-018-0401-z

Bentley, D. R., Balasubramanian, S., Swerdlow, H. P., Smith, G. P., Milton, J., Brown, C. G., ... Smith, A. J. (2008). Accurate whole human genome sequencing using reversible terminator chemistry. *Nature*, *456*(7218), 53–59. doi: 10.1038/nature07517

Boore, J. L. (1999). Animal mitochondrial genomes. *Nucleic Acids Research*, *27*(8), 1767–1780. doi: 10.1093/nar/27.8.1767

Cameron, S. L. (2014). Insect mitochondrial genomics: Implications for evolution and phylogeny. *Annual Review of Entomology*, *59*, 95–117. doi: 10.1146/annurev-ento-011613-162007

Dierckxsens, N., Mardulyn, P., & Smits, G. (2017). Novoplasty: de novo assembly of organelle genomes from whole genome data. *Nucleic Acids Research*,

45(4), e18. doi: 10.1093/nar/gkw955

Edgar, R. C. (n.d.). *Uchime in practice.* Retrieved from https://www.drive5.com/usearch/manual7/uchime_practical.html

Edgar, R. C. (2016). Uchime2: improved chimera prediction for amplicon sequencing. *bioRxiv*. Retrieved from https://api.semanticscholar.org/CorpusID:88955007

Edgar, R. C., Haas, B. J., Clemente, J. C., Quince, C., & Knight, R. (2011). Uchime improves sensitivity and speed of chimera detection. *Bioinformatics*, *27*(16), 2194–2200. doi: 10.1093/bioinformatics/btr381

Glenn, T. C. (2011). Field guide to next-generation dna sequencers. *Molecular Ecology Resources*, *11*(5), 759–769. doi: 10.1111/j.1755-0998.2011.03024.x

Gonzalez, J. M., Zimmermann, J., & Saiz-Jimenez, C. (2004, 09). Evaluating putative chimeric sequences from pcr-amplified products. *Bioinformatics*, *21*(3), 333-337. Retrieved from https://doi.org/10.1093/bioinformatics/bti008 doi: 10.1093/bioinformatics/bti008

Gray, M. W. (2012). Mitochondrial evolution. *Cold Spring Harbor perspectives in biology*, *4*. Retrieved from https://doi.org/10.1101/cshperspect.a011403 doi: 10.1101/cshperspect.a011403

Hahn, C., Bachmann, L., & Chevreux, B. (2013). Reconstructing mitochondrial genomes directly from genomic next-generation sequencing reads—a baiting and iterative mapping approach. *Nucleic Acids Research*, *41*(13), e129. doi: 10.1093/nar/gkt371

Jin, J.-J., Yu, W.-B., Yang, J., Song, Y., dePamphilis, C. W., Yi, T.-S., & Li, D.-Z. (2020). Getorganelle: a fast and versatile toolkit for accurate de novo assembly of organelle genomes. *Genome Biology*, *21*(1), 241. doi: 10.1186/s13059-020-02154-5

Judo, M. S. B., Wedel, W. R., & Wilson, B. H. (1998). Stimulation and suppression of pcr-mediated recombination. *Nucleic Acids Research*, *26*(7), 1819–1825. doi: 10.1093/nar/26.7.1819

Labrador, K., Agmata, A., Palermo, J. D., Ravago-Gotanco, R., & Pante, M. J. (2021). Mitochondrial dna reveals genetically structured haplogroups of bali sardinella (sardinella lemuru) in philippine waters. *Regional Studies in Marine Science*, *41*, 101588. doi: 10.1016/j.rsma.2020.101588

Li, H. (2018, 05). Minimap2: pairwise alignment for nucleotide sequences. *Bioinformatics*, *34*(18), 3094-3100. Retrieved from `https://doi.org/10.1093/bioinformatics/bty191` doi: 10.1093/bioinformatics/bty191

Liang, Q., Bible, P. W., Liu, Y., Zou, B., & Wei, L. (2020, 02). Deepmicrobes: taxonomic classification for metagenomics with deep learning. *NAR Genomics and Bioinformatics*, *2*(1), lqaa009. Retrieved from `https://doi.org/10.1093/nargab/lqaa009` doi: 10.1093/nargab/lqaa009

Metzker, M. L. (2010). Sequencing technologies — the next generation. *Nature Reviews Genetics*, *11*(1), 31–46. doi: 10.1038/nrg2626

Mysara, M., Saeys, Y., Leys, N., Raes, J., & Monsieurs, P. (2015). Catch, an ensemble classifier for chimera detection in 16s rrna sequencing studies. *Applied and Environmental Microbiology*, *81*(5), 1573-1584. Retrieved from `https://journals.asm.org/doi/abs/10.1128/aem.02896-14` doi: 10.1128/AEM.02896-14

Peccoud, J., Lequime, S., Moltini-Conclois, I., Giraud, I., Lambrechts, L., & Gilbert, C. (2018, 04). A survey of virus recombination uncovers canonical features of artificial chimeras generated during deep sequencing library preparation. *G3 Genes—Genomes—Genetics*, *8*(4), 1129-1138. Retrieved from `https://doi.org/10.1534/g3.117.300468` doi: 10.1534/

g3.117.300468

Qin, Y., Wu, L., Zhang, Q., Wen, C., Nostrand, J. D. V., Ning, D., ... Zhou, J. (2023). Effects of error, chimera, bias, and gc content on the accuracy of amplicon sequencing. *mSystems*, *8*(6), e01025-23. Retrieved from `https://journals.asm.org/doi/abs/10.1128/msystems.01025-23` doi: 10.1128/msystems.01025-23

Qiu, X., Wu, L., Huang, H., McDonel, P. E., Palumbo, A. V., Tiedje, J. M., & Zhou, J. (2001). Evaluation of pcr-generated chimeras, mutations, and heteroduplexes with 16s rrna gene-based cloning. *Applied and Environmental Microbiology*, *67*(2), 880–887. doi: 10.1128/AEM.67.2.880-887.2001

Ren, J., Song, K., Deng, C., Ahlgren, N., Fuhrman, J., Li, Y., ... Sun, F. (2020, 01). Identifying viruses from metagenomic data using deep learning. *Quantitative Biology*, *8*. doi: 10.1007/s40484-019-0187-4

Rodriguez-Martin, B., Palumbo, E., Marco-Sola, S., Griebel, T., Ribeca, P., Alonso, G., ... Djebali, S. (2017, 01). Chimpipe: Accurate detection of fusion genes and transcription-induced chimeras from rna-seq data. *BMC Genomics*, *18*. doi: 10.1186/s12864-016-3404-9

Rognes, T., Flouri, T., Nichols, B., Quince, C., & Mahé, F. (2016). Vsearch: a versatile open source tool for metagenomics. *PeerJ*, *4*, e2584. doi: 10.7717/peerj.2584

Sedlazeck, F., Rescheneder, P., Smolka, M., Fang, H., Nattestad, M., von Haeseler, A., & Schatz, M. (2018, 06). Accurate detection of complex structural variations using single-molecule sequencing. *Nature Methods*, *15*. doi: 10.1038/s41592-018-0001-7

Sfeir, A., & Symington, L. S. (2015). Microhomology-mediated end joining: A back-up survival mechanism or dedicated pathway? *Trends in Biochemical*

Sciences, *40*(11), 701-714. Retrieved from https://www.sciencedirect.com/science/article/pii/S0968000415001589 doi: https://doi.org/10.1016/j.tibs.2015.08.006

Vervier, K., Mahé, P., Tournoud, M., Veyrieras, J.-B., & Vert, J.-P. (2015, 11). Large-scale machine learning for metagenomics sequence classification. *Bioinformatics*, *32*(7), 1023-1032. Retrieved from https://doi.org/10.1093/bioinformatics/btv683 doi: 10.1093/bioinformatics/btv683

Willette, D., Bognot, E., Mutia, M. T., & Santos, M. (2011). *Biology and ecology of sardines in the philippines: A review* (Vol. 13; Tech. Rep. No. 1). NFRDI Technical Paper Series. Retrieved from https://nfrdi.da.gov.ph/tpjf/etc/Willette%20et%20al.%20Sardines%20Review.pdf